Enhancing Weak Supervision with Foundation Models: Empowering Expert Knowledge Expression

By

Peilin Yu

Thesis Proposal

Computer Science Department, Brown University

PROVIDENCE, RHODE ISLAND

Fall 2024

Abstract

This thesis proposal advances weak supervision in machine learning by introducing techniques that leverage large-scale foundation models—specifically large language models (LLMs) and vision-language models (VLMs)—to address fundamental challenges in programmatic labeling and model generalization. The proposed work builds on three key methodological contributions: (1) a probabilistic generative model that allows noisy labeling functions to output subsets of possible class labels, instead of being restricted to a single label. This approach expands the scope of heuristics that can be used in developing weak supervision sources, broadening the utilization of expert knowledge; (2) a compositional soft prompting technique, $\mathbb{CSP}$, that treats attributes and objects as learnable vocabulary tokens. This method enhances vision-language models like CLIP, enabling them to generalize across unseen attribute-object combinations without requiring task-specific training data; (3) Alfred, a system that leverages natural language prompts with foundation models to streamline training data curation within programmatic weak supervision frameworks. Building on these initial results, I plan to explore how combining external knowledge with retrieval techniques can further improve the efficiency of prompted weak supervision systems. I aim to develop a prototype agentic system that reduces the human workload in prompted weak supervision by further automating and optimizing the labeling process. The proposed research will conduct empirical validation across diverse domains and tasks, particularly in domains where large-scale labeled data is scarce or expensive to obtain.

Contents

CHAPTER 1

Introduction

In the field of machine learning, a fundamental challenge persists despite significant algorithmic and computational advances: the acquisition of high-quality labeled data. This dependency on extensive labeled datasets has emerged as a critical bottleneck in developing and deploying machine learning systems, particularly in specialized domains where expert knowledge is essential for accurate labeling. Obtaining such data remains prohibitively expensive and time-consuming, often limiting both the scope and impact of machine learning applications.

Weak supervision offers a promising solution by enabling models to learn from noisy, imprecise labels generated by heuristic functions or domain experts, thereby reducing the need for large labeled datasets. Programmatic weak supervision (Ratner et al., 2016a, 2017) formalizes this process by introducing *labeling functions* as a means to encode expert knowledge and heuristic patterns for supervision. These *labeling functions*, which encapsulate domain-specific insights, are combined within a probabilistic model to estimate the underlying true labels, which are subsequently used to train a target model. The integration of domain expertise is essential, as experts possess nuanced knowledge that can inform and enhance labeling functions, making weak supervision more robust and applicable across diverse tasks. However, despite this potential, current approaches for

incorporating expert knowledge in weak supervision face three fundamental challenges:

1. **Limited Expressiveness of Labeling Functions:** Conventional frameworks restrict *labeling functions* to single-class decisions, forcing experts to artificially decompose their nuanced knowledge into simplified rules. This constraint makes the process inefficient and fails to capture the natural complexity of domain expertise.

2. **Limited Model Capability to Handle Compositional Knowledge:** Existing models struggle with compositional knowledge—the ability to recognize novel combinations of known concepts (e.g., "ripe tomato" or "old tiger"). This limitation necessitates acquiring separate annotations for each possible combination, making it impractical to scale as the space of possibilities grows exponentially.

3. **Technical Barriers to Knowledge Transfer:** Current systems often require domain experts to have programming skills to encode their knowledge into rigid, code-based labeling functions, posing a significant barrier. This requirement may lead to underutilization of expert knowledge and increases the manual workload for those who can bridge this gap.

Addressing these challenges is non-trivial due to the need to balance the expressiveness of *labeling functions* with the complexity of modeling their outputs, enhance model generalization without exhaustive annotations, and reduce technical barriers for domain experts. This thesis proposal includes research to address these fundamental challenges by integrating foundation models—specifically, large language models (LLMs) and vision-language models (VLMs)—with enhanced weak supervision techniques:

- **Probabilistic Generative Model for Noisy Partial Labelers (Chapter 2):** I introduce a novel probabilistic generative model that allows *labeling functions* to output subsets of possible class labels, rather than being limited to single-class outputs. This approach increases the flexibility of *labeling functions* and better captures the natural complexity of domain expertise, reducing the need to create an

exhaustive set of simplified labeling functions.

- $\mathbb{CSP}$: **Compositional Soft Prompting (Chapter 3):** I develop a compositional soft prompting technique that treats attributes and objects as learnable vocabulary tokens within vision-language models like CLIP(Radford et al., 2021a). This method enhances the model's capability to handle compositional knowledge by enabling it to recognize novel combinations of known concepts without requiring separate annotations for each possible combination, thus improving scalability and adaptability.

- **Alfred: A Prompted Weak Supervision System (Chapter 4):** I present *Alfred*, a prototype system that leverages natural language prompts with foundation models to simplify the creation of training data in prompted weak supervision frameworks (Smith et al., 2022). This system reduces technical barriers for domain experts by allowing them to encode their knowledge without programming skills, thereby decreasing manual workload and enhancing the utilization of expert knowledge.

Building on these contributions, I will construct Alfred-Apps as a standard benchmark for prompted weak supervision and as a test bed for evaluating and researching future new techniques. With this platform, I plan to explore how integrating external knowledge with retrieval techniques can further enhance the efficiency of prompted weak supervision systems. Additionally, I aim to develop a prototype LLM-based *agentic system* that automates and optimizes the labeling process to further reduce the need for human input while maintaining accuracy and reliability.

Through this research, I seek to advance weak supervision approaches by leveraging foundation models to bridge the gap between expert knowledge and machine learning systems. This work aims to provide more effective mechanisms for knowledge transfer in domains where labeled data is limited or costly, offering new approaches to capturing and utilizing domain expertise within machine learning systems.

CHAPTER 2

Learning from Multiple Noisy Partial Labelers

2.1 Introduction

The need for large-scale labeled datasets has driven recent research on methods for *programmatic weak supervision* (PWS), such as data programming (Ratner et al., 2016a, 2017), adversarial label learning (Arachie and Huang, 2021), learning rules from labeled exemplars (Awasthi et al., 2020), and weak supervision with self-training (Karamanolakis et al., 2021). In PWS, *labeling functions*, such as user-written rules and other heuristics, provide votes on the true labels for unlabeled examples or abstain from voting. Then, a label model, such as a probabilistic generative model or minimax game, is often used to estimate the true labels in a way that accounts for unknown differences in accuracies and other properties of the labeling functions. Finally, these estimated labels are used to train an end model on the unlabeled data to generalize beyond the information contained in the labeling functions. This approach has had recent success with applications in natural language processing (Mallinar et al., 2019; Safranchik et al., 2020), computer vision (Chen et al., 2019), medicine (Fries et al., 2019; Saab et al., 2020; Fries et al., 2021), and the

Web (Bach et al., 2019). However, all of these methods assume that labeling functions cast votes for *individual* classes. In this work, we propose to generalize PWS to support labeling functions that cast votes for a *subset* of classes, called partial labels. We refer to such labeling functions as *partial labeling functions* (PLFs). Our goal is to aggregate information from multiple partial labeling functions that are noisy (i.e., have imperfect accuracy) in order to estimate labels for unlabeled data.

Incorporating partial labels into PWS would enable users to take advantage of a wider range of domain knowledge. In typical PWS frameworks, only heuristics that are specific to one class can be incorporated. As a result, creating labeling functions requires careful task-specific engineering to avoid features that are shared by more than one class. For example, consider the task of classifying images of animals with a label from the set {`HORSE, TIGER, LION, ZEBRA`}. There are many useful heuristics that can be learned from other labeled data sets, such as detectors for claws or stripes (Lampert et al., 2009). Such heuristics divide the label space with multiple partitions. A claw detector could produce two partial labels: {`TIGER, LION`} if a claw is detected and {`HORSE, ZEBRA`} if not. Likewise, a stripe detector could output {`TIGER, ZEBRA`} if stripes are detected and {`HORSE, LION`} if not (Figure 2.1). However, these heuristics cannot be used as labeling functions in current PWS frameworks. More generally, we observe a need for partial labeling functions in many multiclass applications where users want to express heuristics that narrow down the set of possible class labels but are not specific to a single class.

Learning from multiple noisy PLFs is challenging because we must resolve ambiguity arising from three sources: (1) PLF imprecision, i.e., voting for a set of classes instead of a single class, (2) PLF inaccuracy, i.e., voting for a set of classes that does not contain the true class, and (3) conflict among multiple PLFs. A further requirement is that PWS frameworks should support labeling functions that abstain, meaning they can choose not to label certain examples. This is particularly critical for hand-engineered rules that might be highly specialized. A framework for learning from multiple noisy PLFs should therefore

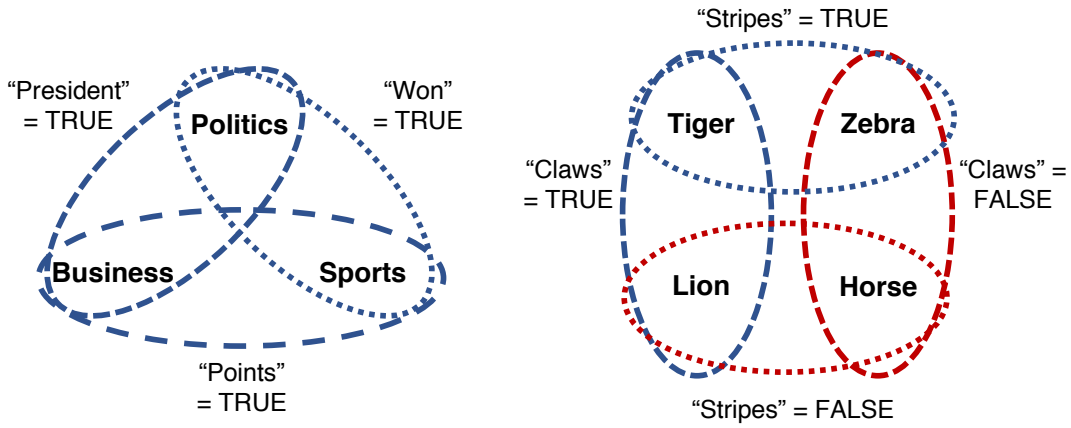

Figure 2.1: Examples of the expressivity of partial labeling functions. On the left, three functions each vote for two of three classes if they observe a particular token in a news article and abstain otherwise. On the right, two functions each vote for two of four classes if they detect a particular attribute in an image. Otherwise, they vote for the other two.

be able to resolve all these types of ambiguities in a principled way while also maintaining the expressive capabilities of existing PWS frameworks.

This problem setting is quite general and is related to multiple lines of work in machine learning, although each of them only addresses part of the problem considered here. As mentioned above, previous PWS frameworks generally require labeling functions to provide a single class label (Ratner et al., 2016a; **?**; Arachie and Huang, 2021; Awasthi et al., 2020; Karamanolakis et al., 2021; Safranchik et al., 2020). One exception is Snorkel MeTaL (Ratner et al., 2019), which is capable of handling labeling functions with a multi-task tree structure where higher-level labeling functions are grouped into super classes that encompass fine-grained classes. This requirement of a tree structure makes modeling partial labels that divide the label space into overlapping subsets practically infeasible. There is also a wide body of work on learning from partial labels, also called superset learning (Jin and Ghahramani, 2002; Nguyen and Caruana, 2008; Luo and Orabona, 2010; Cour et al., 2011; Liu and Dietterich, 2012, 2014; Hüllermeier and Cheng, 2015; Cabannnes et al., 2020; Wang et al., 2020; Cabannes et al., 2021; Zhang et al., 2021b). In these settings, there is generally one partial label per example. The ambiguity in the imprecise labels in such settings can be resolved as a maximum likelihood or risk

minimization problem. Many methods additionally learn the likely confusions between partial and true labels (Durand et al., 2019; Li et al., 2020a; Yan and Guo, 2020; Xie and Huang, 2021). However, such methods do not handle the case of multiple partial labelers that can disagree and abstain. Finally, some work on zero-shot learning (ZSL) creates attribute detectors that can be viewed as partial labelers (Farhadi et al., 2009; Lampert et al., 2009; Palatucci et al., 2009; Jayaraman and Grauman, 2014). PWS with partial labels can also be viewed as a generalization of the transductive ZSL setting (Xian et al., 2018; Wang et al., 2019), in which labelers are allowed to abstain and a class may be associated with multiple attribute values. Across all these areas, there remains a need for learning from multiple noisy partial labelers.

To address this issue, we propose a generalized PWS framework that supports partial labels and handles the additional ambiguity caused by the imprecise outputs of the PLFs. First, we introduce a probabilistic generative model that estimates the agreement between the outputs of each partial labeling function and the latent, true label. Second, we show how to learn the model parameters efficiently for large datasets. For example, we can learn on 100k examples with 10 PLFs in one minute. Since PWS is inherently human-in-the-loop, fast iteration is crucial. Third, we prove that this model's parameters are generically identifiable up to label swapping under mild conditions on the PLFs. This result means that we can estimate the accuracy of each partial labeling function without access to ground truth labels in a principled way. Using the learned parameters, we can compute the posterior distribution over true labels for each example. These probabilistic training labels can then be used to train an end model in the same manner as other PWS frameworks.

We demonstrate this framework with experiments on three text and six object classification tasks. On the text classification tasks, we show that the additional flexibility provided by partial labelers enables heuristics that significantly improve over single-class labelers alone. We find an average 8.6 percentage point improvement in accuracy. On the object classification tasks, we find that modeling the accuracies of the PLFs explicitly enables

us to achieve accuracy comparable to recent embedding-based ZSL methods using only pre-trained attribute detectors. These results provide a foundation for constructing and learning more modular, reusable knowledge sources for weak supervision.

2.2 Related Work

In the past few years, programmatic weak supervision (PWS) has emerged as a systematic approach to efficiently create labeled training data (Ratner et al., 2016a, 2017; Arachie and Huang, 2021; Awasthi et al., 2020; Karamanolakis et al., 2021). A typical PWS framework consists of three stages. First, domain experts engineer weak supervision sources in the form of labeling functions, such as rules or classifiers related to the target task. Second, a label model, such as a probabilistic generative model, is used to estimate the latent true labels using the labeling function outputs. Third, the estimated labels are used to train an end model that generalizes beyond the information in the supervision sources. The core of a typical PWS framework is the label modeling stage. The choice of label model determines what types of supervision sources are supported. Many frameworks are based on crowdsourcing methods (Dawid and Skene, 1979; Nitzan and Paroush, 1982; Gao and Zhou, 2013), where providing a single label is a natural assumption. In the original data programming framework (Ratner et al., 2016a), labeling functions can output a single label or abstain. The label model is generative, meaning that each true label is a latent variable and the observed votes of the labeling functions are conditioned on the true labels. The parameters of the label model are learned by maximizing the marginal likelihood of the observed votes. Statistical dependencies such as correlations among the votes can be modeled, and methods exist to learn specific types of dependencies from unlabeled data (Bach et al., 2017; Varma et al., 2019).

The Snorkel MeTaL (Ratner et al., 2019) framework extends data programming to learn across multiple, related tasks organized in a tree structure. For example, in a fine-grained named entity recognition task, one might use a set of labeling functions that vote on

coarse-grained entity types and separate sets of labeling functions to further vote on the subtypes within each coarse type. The outputs of labeling functions at higher levels of the tree can be thought of as a restricted form of partial labels, in the sense that all labeling functions must follow the same tree-structured organization of the classes. In contrast, in our setting, each partial labeling function can organize the classes into its own, possibly overlapping groups.

Other PWS frameworks have approached labeling functions and the label modeling processes in different ways, but all so far assume that each labeling function votes for a single class. Adversarial label learning (Arachie and Huang, 2021), performance-guaranteed majority vote (Mazzetto et al., 2021b), adversarial multi class learning (Mazzetto et al., 2021a), and related work in semi-supervised ensemble learning (Balsubramani and Freund, 2015, 2016b,a) solve minimax games based on assumed or estimated constraints on labeling function accuracies. Awasthi et al. (2020) proposed learning from rules and exemplars for those rules, learning to downweight the confidence in the rules on data instances not similar to the exemplars. Karamanolakis et al. (2021) proposed integrating PWS with semi-supervised self-training.

Other work on learning with partial labels has focused on the case where there is a single partial label per example. Classifiers can be learned via maximum likelihood estimation (Jin and Ghahramani, 2002; Liu and Dietterich, 2012) or empirical risk minimization (Nguyen and Caruana, 2008; Luo and Orabona, 2010; Cour et al., 2011; Liu and Dietterich, 2014; Hüllermeier and Cheng, 2015; Cabannnes et al., 2020; Cabannes et al., 2021; Feng et al., 2020b). Many methods additionally learn the likely confusions between partial and true labels (Durand et al., 2019; Li et al., 2020a; Yan and Guo, 2020; Xie and Huang, 2021). Wang et al. (2020) proposed learning multiple partially labeled tasks simultaneously, in order to exploit structure among the tasks, but during training there is still only one partial label per prediction. Partial labels are also related to complementary labels (Ishida et al., 2017; Feng et al., 2020a), which are annotations

that indicate which label the example does *not* have.

Our problem setting is also related to some forms of zero-shot learning (ZSL) (Xian et al., 2018; Wang et al., 2019). In zero-shot classification, a model learns to match semantic descriptions of classes to examples of those classes. Once learned, the model can be applied to novel classes. Many early approaches to ZSL created detectors for different attributes (Farhadi et al., 2009; Lampert et al., 2009; Palatucci et al., 2009; Jayaraman and Grauman, 2014). In the transductive setting (Xian et al., 2018; Wang et al., 2019), in which the target classes are known and unlabeled examples of them are available during model development, these detectors can be viewed as restricted partial labeling functions that always divide the label set into non-overlapping groups and never abstain. More recently, much work on ZSL has moved away from relying entirely on attribute detectors, and recent work can be grouped into either embedding-based or generative-based methods (Pourpanah et al., 2020). Embedding-based methods align representation spaces between classes and examples in order to classify unlabeled data (Socher et al., 2013; Frome et al., 2013; Romera-Paredes and Torr, 2015; Xian et al., 2016, 2018; Wang et al., 2019). Some work, e.g., Liu et al. (2019) and Liu et al. (2020), also learn to exploit and expand attribute-based information, but generally still do not use separate attribute detectors. On the other hand, generative-based ZSL methods generate examples of the unseen classes with deep generative models and then train a classifier with that data (Bucher et al., 2017; Verma et al., 2018; Felix et al., 2018; Sariyildiz and Cinbis, 2019; Xian et al., 2019; Narayan et al., 2020). In our experiments, we compare with transductive embedding-based methods, which are more similar to PWS because both involve trying to label a fixed, unlabeled data set. We leave incorporating zero-shot data generation into PWS for future work.

2.3 A framework for Partial Labeling Functions

Following prior work in PWS (Ratner et al., 2016a, 2017), our framework consists of three stages. First, we define partial labeling functions as sources of weak supervision (Section 2.3.1). Second, we propose and analyze a label model to aggregate their outputs (Section 2.3.2). Third, the learned label model is used to compute the posterior distribution for the true label of each unlabeled example, which is used to train a noise-aware classifier (Section 2.3.4).

2.3.1 Partial Labeling Functions

We propose generalizing labeling functions to *partial labeling functions* (PLF) in order to make use of many available weak supervision sources that are informative but not specific enough to identify a single class. PLFs can range in granularity, from dividing the label space into two large groups down to identifying a specific class, i.e., a regular labeling function. This flexibility allows users to take advantage of many additional supervision signals, as we illustrate in our experiments.

A PLF G is a function that maps an unlabeled example to a proper subset of the possible labels or abstains by outputting the full set of all possible labels. Formally, our goal is to learn a classifier $C : \mathcal{X} \to \mathcal{Y}$, where $\mathcal{X}$ is the space of inputs and $\mathcal{Y} = \{y_1, \ldots, y_k\}$ is the set of possible labels. A PLF is then a function $G : \mathcal{X} \to \mathcal{G} \subseteq \mathbb{P}(\mathcal{Y}) \setminus \{\emptyset\}$, where $\mathbb{P}(\mathcal{Y})$ is the power set of $\mathcal{Y}$. $G(X)$ is a partial label for $X \in \mathcal{X}$, i.e., the set of labels that the PLF indicates the example X could have (although this information could be incorrect). If $G(X) = \mathcal{Y}$, the PLF is said to abstain, because it provides no information about the true label. As described further in Section 2.3.2, a key characteristic of a PLF is its codomain $\mathcal{G}$, excluding when the PLF abstains. We denote this set of partial labels for a PLF G as $T(G) = \mathcal{G} \setminus \{\mathcal{Y}\}$. To ensure that our label model is well-defined, we will impose these conditions on $T(G)$: (1) each label $y \in \mathcal{Y}$ appears in at least one element of $T(G)$, and (2) no label $y \in \mathcal{Y}$ appears in every element of $T(G)$. These are very mild

conditions that can easily be satisfied by adding a "dummy" output to the codomain $\mathcal{G}$ that the PLF might not actually produce. A PLF can be defined based on a variety of noisy supervision heuristics using domain knowledge and/or available resources, such as classifiers for related tasks. To better understand PLFs, consider the following text and object classification examples corresponding to Figure 2.1:

Example 1. Consider a news classification task where

$$\mathcal{Y} = \{\texttt{POLITICS, SPORTS, BUSINESS}\}$$

. In this task, some words can be very informative as supervision sources even if they do not narrow the example down to a specific class. For example, the word "president" may frequently appear in both political and business contexts. We can construct a PLF G based on a simple token matcher for "president" such that $G : \mathcal{X} \to \{\{\texttt{BUSINESS}, \texttt{POLITICS}\}, \{\texttt{SPORTS}\}, \mathcal{Y}\}$. If the token "president" appears in the example X, then $G(X) = \{\texttt{BUSINESS, POLITICS}\}$. Otherwise, $G(X) = \mathcal{Y}$, i.e., G abstains, because the absence of the token is not enough to conclude anything about the label with high confidence. In this example, $T(G) = \{\{\texttt{BUSINESS}, \texttt{POLITICS}\}, \{\texttt{SPORTS}\}\}$. Notice here $\{\texttt{SPORTS}\}$ is a "dummy" label set to satisfy the conditions on $T(G)$ described above.

Example 2. Consider an object classification task where

$$\mathcal{Y} = \{\texttt{HORSE}, \texttt{TIGER}, \texttt{LION}, \texttt{ZEBRA}\}$$

. Following work in zero-shot learning, we can build a binary classifier for the visual attribute of having stripes by training on other classes of animals for which we already have labels. We can then use the classifier's output to define a PLF $G_1 : \mathcal{X} \to \{\{\texttt{TIGER}, \texttt{ZEBRA}\}, \{\texttt{HORSE}, \texttt{LION}\}\}$. For an example X, if the stripes detector returns a positive label, then $G_1(X) = \{\texttt{TIGER}, \texttt{ZEBRA}\}$. Otherwise, $G_1(X) = \{\texttt{HORSE}, \texttt{LION}\}$. We can similarly construct a PLF with a claw detector as $G_2 : \mathcal{X} \to \{\{\texttt{TIGER}, \texttt{LION}\}, \{\texttt{HORSE}, \texttt{ZEBRA}\}\}$.

PLFs are a generalization of the labeling functions used in prior work on PWS; traditional labeling functions can be represented as PLFs with codomain $\mathcal{G} = \{\{y_1\}, \ldots, \{y_k\}, \mathcal{Y}\}$. PLFs provide the additional flexibility to users of incorporating weak supervision heuristics with differing granularitities and ways of dividing the label space.

2.3.2 Label Model

In our framework, users provide two inputs: PLFs and unlabeled examples in $\mathcal{X}$. Like other PWS frameworks, at the core of our method is a probabilistic label model that captures the properties of the weak supervision sources by representing the unknown ground-truth labels as latent variables. In this subsection, we propose and analyze a probabilistic label model for PLFs. Our label model is straightforward to define as a generalization of existing ones for labelers that provide a single label (Dawid and Skene, 1979; Ratner et al., 2016a). It defines a correct output of a partial labeler as one that is consistent with the unknown, true label, i.e., that the true label is in the output set. If the partial labeler is trivially correct because it outputs the set of all possible labels, it is said to abstain, which is not counted towards its estimated accuracy.

While straightforward to define, using the model introduces new challenges. The first challenge is practical. Existing methods for optimizing the marginal likelihood of the label model in the single-label case do not work for PLFs. The matrix factorization approach proposed by Ratner et al. (2019) exploits the fact that the matrix of agreements among labelers can be expressed as a product of the model parameters, but this does not hold in the PLF case. Bach et al. (2019) proposed optimizing the likelihood in an auto-differentiation framework like PyTorch (Paszke et al., 2017). We find that naively applying this approach is impractically slow, and that carefully defining the computation graph leads to a 300× speedup.

The other challenge we address is theoretical. We consider when it is possible to learn the parameters of the model without access to ground truth in a principled way, i.e., when

the model is identifiable. Prior theoretical work either on PWS or learning with partial labels is not applicable to our scenario. Prior work on identifiability for PWS does not consider partial labels (Ratner et al., 2019; Safranchik et al., 2020), and new conditions and arguments are needed to handle them. Prior work on risk bounds for learning with partial labels does not address noise nor multiple labelers (Cour et al., 2011; Liu and Dietterich, 2014; Hüllermeier and Cheng, 2015; Feng et al., 2020c). Therefore, our main contributions in this section are a fast learning technique and a theorem characterizing sufficient identifiability conditions for our model.

Setup For a classification task with input space $\mathcal{X}$ and label space $\mathcal{Y} = \{y_1, \ldots, y_k\}$, we are given m unlabeled examples $X = (X_1, \ldots, X_m)$ with unknown ground truth labels $Y = (Y_1, \ldots, Y_m)$ such that (X, Y) are i.i.d. samples from some distribution $\mathcal{D}$. We are also given n PLFs $G = (G_1, \ldots, G_n)$. We use G as shorthand for the $m \times n$ array of PLF outputs where $G_{ai} = G_i(X_a)$ when it is clear from context.

Joint Distribution We define a joint distribution $P(G, Y)$ over the outputs of the PLFs on X and the latent, true labels Y. Like prior work (Ratner et al., 2016a), we assume that the PLF outputs are conditionally independent given the true labels, i.e., the naive Bayes assumption. In practice this works well, but extending work on learning more complex distributions for other types of PWS is a potential direction for future exploration (Bach et al., 2017; Varma et al., 2019). Analogous to prior work (Ratner et al., 2019), for each PLF G_i, we define parameters $\alpha_i \in [0, 1]^k$ and $\beta_i \in [0, 1]$. Each element α_{ij} is the accuracy of G_i on examples of class y_j, i.e., the probability that $y_j \in G_{ai}$ given that X_a has label y_j and G_{ai} is not $\mathcal{Y}$. β_i is the propensity of G_i voting, i.e., not abstaining. In other words, $\beta_i = P(G_{ai} \neq \mathcal{Y})$. In our framework, the class balance $P(Y)$ can either be a learned distribution or fixed. We assume that if a PLF G_i makes a mistake, it outputs an incorrect partial label from $T(G_i)$ uniformly at random.

To define the joint distribution $P(G, Y)$, for each PLF G_i we also need to refer to the sets in $T(G_i)$ that are consistent or inconsistent with each label. Let $N_{ij} = \{\mathcal{L} | y_j \in \mathcal{L} \text{ for } \mathcal{L} \in$

$T(G_i)\}$ be the set of label sets in the codomain of G_i that contain label y_j (excluding $\mathcal{Y}$). Likewise, let $N_{ij}^{\mathrm{C}} = \{\mathcal{L} | y_j \notin \mathcal{L} \text{ for } \mathcal{L} \in T(G_i)\}$ be the set of label sets in the codomain of G_i that do not contain label y_j. Then, the joint distribution is

$$P(G, Y) = \prod_{a=1}^{m} P(Y_a) \prod_{i=1}^{n} P(G_{ai} | Y_a) \tag{2.1}$$

where

$$P(G_{ai} | Y_a = y_j) = \begin{cases} 1 - \beta_i, & \text{if } G_{ai} = \mathcal{Y} \\ \frac{\beta_i \alpha_{ij}}{|N_{ij}|}, & \text{if } y_j \in G_{ai} \neq \mathcal{Y} \\ \frac{\beta_i (1 - \alpha_{ij})}{|N_{ij}^{\mathrm{C}}|}, & \text{if } y_j \notin G_{ai}. \end{cases} \tag{2.2}$$

Learning Given the unlabeled examples and PLF outputs G, our goal is to estimate the parameters of $P(G, Y)$ (denoted collectively as Θ) and compute the posterior $P(Y|G)$ over the unknown labels. To estimate Θ, we maximize the marginal likelihood of the observed outputs of the PLFs:

$$\hat{\Theta} = \operatorname*{argmax}_{\Theta} P_{\Theta}(G) = \operatorname*{argmax}_{\Theta} \sum_{Y} P_{\Theta}(G, Y) \ . \tag{2.3}$$

This optimization is implemented in PyTorch (Paszke et al., 2017). The marginal log likelihood of a batch of examples is computed in the forward pass, and stochastic gradient descent is used to update the parameters. We find that the way the likelihood computation is implemented in the forward pass can lead to an orders-of-magnitude difference in training time. For every example, we need to compute its conditional likelihood for every class based on votes from every PLF. Naively, this will require three layers of for loops through examples, PLFs, and classes. We can speed up the computation by expressing the conditional log likelihood computation as a sequence of matrix operations. This optimization trades space for speed by precomputing intermediate values and taking advantage of vectorized matrix operations.

Let m be the number of instances in one batch, n be the number of PLFs and k be the number of classes. For each batch we precompute accuracy indicator matrices $\mathbf{AI} \in \{-1, 1\}^{m \times n \times k}$ and count matrices $\mathbf{N} \in \mathbb{Z}^{m \times n \times k}$ where entry $\mathbf{AI}_{a,i,j} = 1, \mathbf{N}_{a,i,j} = -\log |N_{i,j}|$ if class y_j is in the label subset output by the i-th PLF on the a-th example, and $\mathbf{AI}_{a,i,j} = -1, \mathbf{N}_{a,i,j} = -\log |N^{\mathrm{C}}_{i,j}|$ otherwise. We also precompute propensity indicator matrices $\mathbf{PI} \in \{0, 1\}^{m \times n}$ where entry $\mathbf{PI}_{a,i} = 1$ if the a-th instance received a non-abstaining vote (vote is not $\mathcal{Y}$) from the i-th PLF. Let $\mathbf{A} \in \mathbb{R}^{n \times k}$ be the log of the accuracy parameters and $\mathbf{B} \in \mathbb{R}^n$ be the log of the propensity parameters. We can map these parameters back to probability space as

$$\begin{aligned} \alpha_{i,j} &= \frac{\exp(\mathbf{A}_{i,j})}{\exp(\mathbf{A}_{i,j}) + \exp(-\mathbf{A}_{i,j})} \text{ and} \\ \beta_b &= \frac{\exp(\mathbf{B}_i)}{\exp(\mathbf{B}_i) + 1} \, . \end{aligned} \tag{2.4}$$

We extend $\mathbf{PI}$, $\mathbf{A}$, and $\mathbf{B}$ to $\mathbf{PI}^{\text{ext}}$, $\mathbf{A}^{\text{ext}}$, and $\mathbf{B}^{\text{ext}}$ in 3 dimensions, with $\mathbf{PI}$ replicated along the third axis k times, $\mathbf{A}$ replicated along the first axis m times, and $\mathbf{B}$ replicated along the first axis m times and third axis k times. Then, during each forward pass, we only need to calculate normalizing matrices $\mathbf{ZA} \in \mathbb{R}^{n \times k}$ and $\mathbf{ZB} \in \mathbb{R}^n$ for accuracy and propensity respectively where

$$\begin{aligned} \mathbf{ZA}_{i,j} &= -\log\left(\exp(\mathbf{A}_{i,j}) + \exp(-\mathbf{A}_{i,j})\right) \text{ and} \\ \mathbf{ZB}_i &= -\log(\exp(\mathbf{B}_i) + 1) \, . \end{aligned} \tag{2.5}$$

We similarly extend $\mathbf{ZA}$ and $\mathbf{ZB}$ to $\mathbf{ZA}^{\text{ext}}$ and $\mathbf{ZB}^{\text{ext}}$. During the forward pass we calculate the batch conditional log likelihood by first computing a 3-dimension tensor:

$$\underset{m \times n \times k}{\mathbf{T}} = \underset{m \times n \times k}{\mathbf{A}^{\text{ext}}} \odot \underset{m \times n \times k}{\mathbf{AI}} + \underset{m \times n \times k}{\mathbf{N}} + \underset{m \times n \times k}{\mathbf{B}^{\text{ext}}} + \underset{m \times n \times k}{\mathbf{ZA}^{\text{ext}}}$$

and then summing over the n PLFs:

$$\log \underset{m\times k}{P(G|Y)} = \sum_{n} \left(\underset{m\times n\times k}{\mathbf{ZB}^{\text{ext}}} + \underset{m\times n\times k}{\mathbf{PI}^{\text{ext}}} \odot \underset{m\times n\times k}{\mathbf{T}} \right) \tag{2.6}$$

where $\odot$ is element-wise multiplication. The marginal likelihood is then combuted by summing over the k possible classes. This modification allows us to remove for loops in our code from the computation graph, and instead use only optimized matrix operations. This approach leads to a 300× speedup in training time compared to a naive approach. This speedup makes the framework practical for iterative PLF development. For example, learning with 100k examples and 10 PLFs on an Intel i5-6600k CPU takes one minute.

Identifiability An important theoretical question is whether it is reasonable to try to learn the parameters of $P(G, Y)$ even though Y is never observed. We answer this question affirmatively by showing that as long as the codomains of the PLFs are sufficiently targeted or diverse, it is possible to determine the parameters of the label model (up to label swapping) using only the distribution of PLF outputs $P(G)$, except for on a measure zero subset of the space of possible parameter values. This property is the strongest useful notion of identifiability for models with latent variables (Allman et al., 2015). A model whose parameters can be determined except for on a measure zero subset is called *generically identifiable*. *Label swapping* refers to the fact that unobserved classes in a latent variable model can be relabeled without changing the observed distribution. This means that the map going from the observed distribution of a label model with k classes to parameter values is at best $k!$-to-one and cannot be one-to-one even under ideal conditions. In practice, label swapping is not an issue because most PLFs are more accurate than random guessing. We state a condition on the PLF codomains that is sufficient for identifiability in the following theorem.

Theorem 1. The parameters of the model P(G, Y) described in Section 2.3.2 are generically identifiable up to label swapping provided that the collection G of partial

labeling functions can be partitioned into three disjoint non-empty sets S_1, S_2, and S_3 such that, for sets $j = 1, 2$ and all classes $y \in \mathcal{Y}$, we can choose label sets $t_i \in T(G_i)$ satisfying $\bigcap_{G_i \in S_j} t_i = \{y\}$.

2.3.3 Proof of Theorem 1

Theorem 1 provides sufficient conditions for the generic identifiability of the label model described in Section 2.3.2. In this section, we prove this theorem. Our proof is non-trivial because our label model yields probability distributions that are a measure-zero subset of the distributions considered by Allman et al. (2009). Allman et al. (2009) allows each entry of the class-conditional distributions to be any value in the interval $[0, 1]$ such that the entries sum to 1, whereas our label model imposes additional algebraic constraints on the entries. Allman et al. (2009) establishes identifiability except for on a measure-zero subset of the distributions that they consider, but we are unable to directly apply their results because our family of distributions might be contained in the measure-zero subset they exclude. It is therefore necessary to establish that the set of distributions for which identifiability does not exist is of measure zero with respect to the distributions that can be produced by our label model, or, equivalently, the set of values of the accuracies $\alpha_{i,j}$, propensities β_i, and class balance $P(Y)$ for which identifiability does not exist has measure zero with respect to the set of all possible parameter values.

Background The key tool that we use in our proof is Kruskal's unique factorization theorem, which relies on the concept of Kruskal rank (Kruskal, 1977). The *Kruskal rank* of a matrix is defined to be the largest integer n such that every set of n rows is linearly independent. A useful fact is that a matrix with full row rank also has full Kruskal rank. Kruskal's theorem says that if, for $u = 1, 2, 3$, we have a $k \times r_u$ matrix M_u with Kruskal rank I_u, and these I_u satisfy

$$I_1 + I_2 + I_3 \geq 2k + 2 \tag{2.7}$$

then, given only the three-dimensional tensor M where entry (a, b, c) is given by

$$M(a, b, c) = \sum_{j=1}^{k} M_1(j, a) M_2(j, b) M_3(j, c) \tag{2.8}$$

we can recover the original matrices M_u.

Allman et al. (2009) makes the connection that the probability distribution of a latent variable model with three observed variables and one latent variable that takes on a finite set of values can be described by the matrix M. Each M_u can be intrepreted as a conditional probability matrix where row c is a probability distribution over the possible values of feature u given that the latent variable has value c.

In situations where there are more than three observed variables, variables can be combined to form "grouped" variables that satisfy the theorem conditions, if needed. In our case, it is possible that individual PLFs do not have codomains that are large and diverse enough to satisfy the Kruskal rank requirement. In these situations, the condition can be satisfied by amalgamating multiple PLFs to form a grouped PLF, which can be viewed as an observed variable with a codomain that is the Cartesian product of the codomains of its member PLFs.

We show that the conditions of Theorem 1 ensure that, for a generic choice of parameters in a k-class model, there is a tripartion of the PLFs such that two of the corresponding conditional probability matrices have full Kruskal rank (k) and the third conditional probability matrix has a Kruskal rank of at least 2. Thus, the conditions of Kruskal's unique factorization theorem are satisfied, so we can recover the conditional probability matrices, from which the accuracies $\alpha_{i,j}$, propensities β_i, and class balance $P(Y)$ can be computed by solving a system of equations.

Proof of Theorem 1. S_1, S_2, and S_3 partition the PLFs into three disjoint subsets. For $u = 1, 2, 3$, define some ordering to the PLFs in subset S_u so that $S_{u,i}$ gives the i-th PLF in

the subset. We will treat S_u as a "grouped" PLF with codomain $\mathcal{G}(S_u) = \{(t_1, t_2, ..., t_{|S_j|}) \mid t_1 \in \mathcal{G}(S_{u,1}), t_2 \in \mathcal{G}(S_{u,2}), ..., t_{|S_u|} \in \mathcal{G}(S_{u,|S_u|})\}$, where $\mathcal{G}(G)$ denotes the codomain of PLF G. Let M_u denote the $k \times |\mathcal{G}(S_u)|$ conditional probability matrix for the combined output of all PLFs in subset S_u, where each entry is a product containing some combination of β_i, $(1-\beta_i)$, $\alpha_{i,j}$, $(1-\alpha_{i,j})$, and normalizing constants. We assume that the class balance $P(Y)$ has positive entries and all PLFs have non-zero propensities β_i, because any extraneous class labels or non-voting PLFs would be removed. Define $\tilde{M}_1 = \mathsf{diag}(P(Y))M_1$, where $\mathsf{diag}(\mathbf{v})$ denotes the matrix with the entries of vector $\mathbf{v}$ along its main diagonal and zeros elsewhere. $P(G)$, the observed distribution of PLF outputs, corresponds to the three-dimensional tensor obtained from applying Equation 2.8 to $\tilde{M}_1$, M_2, and M_3. We will consider the Kruskal ranks of $\tilde{M}_1$, M_2, and M_3, which we respectively denote I_1, I_2, and I_3.

We first consider M_2. The (row) rank of a matrix A is equal to the largest integer n for which there exists an $n \times n$ submatrix of A that has a nonzero determinant. The determinant of such a submatrix is called an n-minor. M_2 has less than full row rank if and only if all of its k-minors are zero. This condition can be expressed as the nonvanishing of a polynomial in the entries of M_2, which are themselves functions of the label model parameters. In other words, the set of parameter values for which M_2 does not full row rank is the zero set of this polynomial. As described in Allman et al. (2009), so long as the polynomial is not identically zero, the parameter values yielding less than full row rank is a measure-zero subset of the full parameter space. To show that this polynomial is not zero for all values in the parameter space, it is sufficient to show that there exists at least one set of parameter values for which the polynomial is nonzero, or, equivalently, that there is a set of parameter values for which M_2 has full row rank.

The values of the propensities β_i and class balance $P(Y)$ do not affect row rank as long as they are nonnegative, as assumed above. We now show that there is a setting of the accuracies $\alpha_{i,j}$ for which the Kruskal rank of M_2 is k. Set all $\alpha_{i,j} = 1$. By the conditions

of Theorem 1, for each class c, there is an output in the codomain of S_2 for which c appears in all of the individual PLF outputs and no other class appears in all outputs. This implies the following two statements about the column in M_2 that is associated with this output: (1) the c-th entry of this column does not contain $(1 - \alpha_{i,j})$ in its product, and (2) all other entries are products containing at least one $(1 - \alpha_{i,j})$. When $\alpha_{i,j} = 1$, these entries containing $(1 - \alpha_{i,j})$ are all zero. In other words, M_2 has k columns that are all zero except for a single entry, and the row containing this entry is different across the k columns. These columns form the basis of a column space of dimension k. For any matrix A, $\dim(\text{Col } A) = \dim(\text{Row } A) =$ row rank of A. Thus, the row rank of M_2 is k. Since M_2 has full row rank when all $\alpha_{i,j} = 1$, it also has full Kruskal rank. This shows that the polynomial whose nonvanishing determines whether M_2 has full row rank is not identically zero, so M_2 generically has full row and Kruskal rank.

We now consider $\tilde{M}_1$. The arguments applied to M_2 can be applied exactly to M_1, but we are interested in $\tilde{M}_1 = \mathsf{diag}(P(Y))M_1$. However, since we assumed that $P(Y)$ contains only positive entries, and multiplying each row of a matrix by a nonzero scalar does not change its row rank, the same arguments can be applied to $\tilde{M}_1$. We conclude that $\tilde{M}_1$ also generically has full Kruskal rank.

Finally, we consider M_3. The Kruskal rank of a matrix is less than two only if there are two rows that are scalar multiples of each other. This can happen in our model only when the class conditional accuracies for two classes are exactly equal, which corresponds to a measure zero subset of the parameter space. Thus, we can generically assume that M_3 has a Kruskal rank of $I_3 \geq 2$.

Since, generically, M_1 and M_2 have Kruskal ranks of k and M_3 has a Kruskal rank of at least 2, we have $I_1 + I_2 + I_3 \geq 2k + 2$, so Kruskal's unique factorization theorem tells us that we can recover $\tilde{M}_1$, M_2, and M_3 from $P(G)$, the observed distribution of PLF outputs. Once $\tilde{M}_1$, M_2, and M_3 are known, the accuracies $\alpha_{i,j}$, propensities β_i, and class balance $P(Y)$ can be computed using algebraic manipulations. $\square$

This theorem tells us that it is reasonable to try to estimate the PLF accuracies even though the true class labels are never observed. Our proof adapts ideas presented in Theorem 4 of Allman et al. (2009), which uses Kruskal's unique factorization theorem and feature grouping to establish conditions for the generic identifiability of a naive Bayes model with arbitrary parameters. Since the space of models we consider is equivalent to a measure zero subset of the parameters in an arbitrary naive Bayes model, an additional proof is needed to show that these parameters are generically identifiable. We develop a novel argument to show that the above is a sufficient condition for identifiability. We show that for any distribution satisfying the condition described in Theorem 1, we can construct matrices representing factors of the joint distribution over observations that generically have full Kruskal rank. This is sufficient to satisfy the conditions in Kruskal's theorem.

In words, the condition described in Theorem 1 requires that for each class y, we can select a label group from the codomain of each PLF in S_1 such that the intersection of these label groups contains only the class y. This condition also applies to S_2. One way to satisfy this condition is to create PLFs that produce single-class label groups. For example, if PLF G_i contains $\{1\}, \{2\}, ..., \{k\}$ in its codomain, then any set S_j that contains G_i will satisfy the Theorem 1 condition. However, even if no PLFs output any single-class label sets, it is still possible for the label model parameters to be identifiable because the condition can also be satisfied by using multiple PLFs with different codomains. Suppose that we want to show that the condition is satisfied for class 1 and we have $\{1, 2, 3\} \in T(G_1)$, $\{1, 3, 4\} \in T(G_2)$ and $\{1, 2, 4\} \in T(G_3)$. The intersection of these sets is $\{1\}$.

2.3.4 Noise-Aware Classifier

The final stage of our framework is to train a classifier. After $P(G, Y)$ is estimated with unlabeled data, we compute the posterior $P(Y|G)$. Then, we minimize the expected empirical risk with respect to this distribution. For classifiers that output probabilistic

predictions, the loss function becomes the cross-entropy loss weighted by the posterior over true labels. As in other PWS frameworks (Ratner et al., 2016a, 2017, 2019; Safranchik et al., 2020), many off-the-shelf neural networks can be chosen based on the task.

2.4 Experimental Results

We demonstrate benefits of incorporating partial labels into PWS on applications in text and object classification. In Section 2.4.1, we compare our framework with baselines that (1) use only traditional labeling functions and (2) heuristically aggregate partial labels without a probabilistic model. Our proposed approach significantly improves accuracy over both baselines. In Section 2.4.2, we use pretrained visual attribute detectors as PLFs for classifying unseen objects. Our framework achieves accuracy that is competitive with recent embedding-based transductive ZSL methods. While our framework is not designed specifically for ZSL, we present this comparison to demonstrate its flexibility and show another scenario where modeling the noise of multiple partial labeling functions can significantly improve performance relative to a heuristic approach. It shows that discrete attribute detectors can be competitive with recent ZSL approaches. These two sets of experiments are complementary, showing that our framework benefits both hand-engineered rules and partial labeling schemes defined in prior work.

We make available the code for our framework[1] and experiments.[2]

2.4.1 Text Classification

We first evaluate the benefit of incorporating PLFs into text classification with hand-engineered rules.

Datasets We consider three datasets. First, SciCite (Cohan et al., 2019) is a citation classification dataset sourced from scientific literature. The corresponding task is to classify

[1] `github.com/BatsResearch/nplm`
[2] `github.com/BatsResearch/yu-aistats22-code`

	SciCite		TREC-6		AG-News	
	ACC	F1	ACC	F1	ACC	F1
Supervised (Dev. Set)	78.5±1.1	75.5±1.5	88.3±1.6	76.2±2.0	80.1±1.7	79.9±1.9
LFs Only (w/o End)	65.1	44.7	22.0	23.8	33.0	25.1
NC (w/o End)	73.2	69.2	29.6	32.6	46.1	43.5
NPLM (w/o End)	71.5	69.4	38.2	43.0	51.1	49.4
LFs Only	78.6±0.7	76.7±0.8	67.2±0.9	69.3±1.0	79.8±0.9	79.6±0.8
NC	80.2±0.6	77.7±0.6	67.8±1.5	56.5±0.3	78.0±1.0	77.2±1.0
NPLM	81.4±1.3	79.5±1.3	85.0±0.8	85.7±0.7	85.0±0.5	84.7±0.5
NPLM vs. LFs Only	↑ 2.8	↑ 2.8	↑ 17.8	↑ 16.4	↑ 5.2	↑ 5.1
NPLM vs. NC	↑ 1.2	↑ 1.8	↑ 17.2	↑ 29.2	↑ 7.0	↑ 7.5

Table 2.1: Results for text classification with mean accuracy (ACC), macro F1 (F1) and 95% CIs.

```
def first_word_PLF(instance):
    '''
    Label by first word of question.
    ABBR - Abbreviation
    DESC - Description and concepts
    ENTY - Entities
    HUM - Human beings
    LOC - Locations
    NUM - Numeric values
    '''
    word = instance.split()[0].lower()
    if word == "who": return [HUM]
    elif word == "where": return [LOC]
    elif word == "when": return [NUM]
    elif word == "why": return [DESC]
    elif word == "how": return [DESC, NUM]
    elif word == "name": return [ENTY, HUM]
    else: return [ABBR, DESC, ENTY,
                  HUM, LOC, NUM]   # Abstain
```

Figure 2.2: A partial labeling function developed for the TREC-6 question-type task. It uses the first word of the sentence to possibly narrow down the set of labels.

a citation as referring to either background information, method details, or results. Second, TREC-6 (Li and Roth, 2002) is a question classification dataset containing open-domain questions. The task is to classifiy each question as asking about one of six semantic categories. Finally, AG-News (Gulli, 2005) is a large-scale news dataset. The task is to classify each example as one of four topics.

PLF Development

For text tasks, we develop PLFs by inspecting examples from the development set for each dataset (916 examples for SciCite, 389 for TREC-6, and 500 for AG-News). We

implement heuristic rules as Python functions that take as input the example text and any available metadata (such as the name of the section in which the sentence appears for SciCite). Most rules rely on checking for keywords or other surface patterns in the text. Figure 2.2 shows an example for TREC-6, in which we use the first word of a question to vote on what type of question it is. This example illustrates some of the utility of PLFs. Some words like "who" are sufficient to reliabily identify a single class. Others like "how" greatly narrow the set of possible lables, even though they do not specify a single label.

Methods We evaluate methods for aggregating the outputs of the PLFs to train an end model. First, as a baseline, we consider using only the PLFs that are equivalent to traditional labeling functions, i.e., they always output one label or abstain. We call this baseline **LFs Only**. Second, as another baseline we use a heuristic called **Nearest Class (NC)**, which chooses the class with the highest number of compatible partial labels. This baseline is a generalized majority vote heuristic for PLFs. Finally, our method, called **Noisy Partial Label Model (NPLM)**, is our label model from Section 2.3.2. In all cases, we use the estimated labels to fine-tune a pretrained BERT (Devlin et al., 2019) base uncased English model with a three-layer classification head. Following prior work (Ratner et al., 2016a, 2017), we use the expected cross entropy w.r.t. $P(Y|G)$ as our training loss. As ablations, we also report performance using the aggregated PLF outputs directly as predictions, without training an end model, denoted **(w/o End)**.

Results We report mean micro-averaged accuracy and macro-averaged F1 of the compared methods in Table 2.1 on the standard test sets. Results using the end model are shown with 95% confidence intervals obtained using five different random seeds. NPLM consistently improves F1 and accuracy relative to LFs Only (8.6 and 8.1 percentage points on average, respectively) and NC (8.5 and 12.8 percentage points on average, respectively). The performance advantage over LFs Only demonstrates the benefits of additional weak supervision that can be expressed as PLFs, and the advantage over NC demonstrates that the proposed label model is learning useful information. The ablated versions of

the methods significantly underperform their counterparts, showing that in all cases the end model learns to generalize beyond the information contained in the weak supervision heuristics. Many of the errors are on examples for which all supervision sources abstain, where a label is chosen arbitrarily or according to the class prior $P(Y)$. For context, we also report the performance of the end model trained on the development set with ground-truth labels. NPLM significantly outperforms it in most cases. On TREC-6, the supervised baseline has a higher accuracy but much lower macro-averaged F1, indicating that our method does significantly better on the rarer classes.

	LAD (ACC)						AwA2 (MCA)		
	Animals	Fruit	Vehicles	Electronics	Hairstyles	Avg.	U	S	$\mathcal{H}$
ConSE—Norouzi et al. (2013)	36.9	29.8	37.5	28.3	24.6	31.4	0.5	90.6	1.0
ESZSL—Romera-Paredes and Torr (2015)	50.2	37.2	45.8	32.8	31.8	39.6	77.8	5.9	11.0
SynC—Changpinyo et al. (2016)	61.6	51.4	54.9	43.0	29.1	48.0	90.5	10.0	18.0
VCL—Wan et al. (2019)	75.4± 0.8	35.0± 1.0	62.4± 0.5	36.7± 0.5	33.8± 0.7	48.7± 0.3	21.4	89.6	34.6
QFSL—Song et al. (2018)	-	-	-	-	-	-	66.2	93.1	77.4
WDVSc—Wan et al. (2019)	97.2± 0.8	43.3± 1.3	82.1± 0.6	54.8± 1.1	31.1± 2.6	61.7± 0.6	76.4	88.1	81.8
NC (w/o End)	65.8	31.2	60.3	40.3	39.1	47.3	47.7	-	-
NPLM (w/o End)	86.0	38.7	73.5	51.8	45.9	59.2	68.2	-	-
NC	71.9±1.2	36.2±0.6	65.3±1.2	48.0±0.7	40.9±0.5	52.5±0.3	43.1± 1.2	91.8± 0.2	58.6± 1.1
NPLM	87.6± 0.2	42.4± 0.8	77.0± 0.2	57.7± 0.7	46.9± 0.9	62.3± 0.2	71.1± 0.6	91.9± 0.1	80.1± 0.3
NPLM vs. NC	↑ 15.7	↑ 6.2	↑ 11.7	↑ 9.7	↑ 6.0	↑ 9.8	↑ 28.0	-	↑ 21.5

Table 2.2: Results for object classification. For LAD, we report mean accuracy (ACC) with 95% CIs across the five standard splits for each of the five subtasks. For AWA2, we report mean class accuracy (MCA) with 95% CIs. We evaluate AWA2 in a generalized setting: S and U denotes MCA on the seen and unseen classes respectively. $\mathcal{H}$ is the harmonic mean.

2.4.2 Object Classification

In this task, we show how our framework can be used to model discrete visual attribute detectors, and that this approach can achieve results competitive with recent embedding-based ZSL methods. Although they have not been used often in recent ZSL work, discrete attribute detectors have benefits such as modularity and interpretability. These experiments show that modeling them as PLFs with our unsupervised label model can lead to good accuracy.

Datasets We consider the Large-Scale Attribute Dataset (LAD) (Zhao et al., 2019) and Animals with Attributes 2 (AwA2) (Xian et al., 2018), which both provide class-level

discrete visual attributes. LAD is a recently proposed attribute-based dataset with 78k instances that organizes common objects into five sub-datasets: electronics, vehicles, fruits, hairstyles and animals. For each sub-dataset, the classes are divided into five folds of seen and unseen classes, and average performance over all tasks is used as a benchmark for ZSL. AwA2 is a popular ZSL animal classification dataset consisting of ∼30k instances with 85 binary attributes, 40 seen classes, and 10 unseen classes.

PLF Development Following early work on zero-shot object classifcation (Farhadi et al., 2009; Lampert et al., 2009; Palatucci et al., 2009; Jayaraman and Grauman, 2014), we model each visual attribute in the datasets with a binary classifier. In all cases, the classifiers are trained on the seen classes for that task or fold, and the unseen classes are not used at all, not even as validation data. To create classifiers for LAD, we extract features from a ResNet-50 (He et al., 2016) pretrained on ILSVRC (Russakovsky et al., 2015), in order to compare fairly with prior work. For AwA2, we fine-tune a pretrained ResNet-101 on the seen classes. Each class is trained with respect to the class-wise attribute annotations on the training sets of the seen classes. We define the PLFs according to the provided attribute annotations from the respective datasets.

Methods We incorporate PLFs into the NC and NPLM methods as described in Section 2.4.1. We use examples of the unseen classes as our unlabeled data. In all cases, our end models are three-layer perceptrons trained on the extracted features (ResNet-50 for LAD and ResNet-101 for AwA2). As with the text data, we use the expected cross entropy with respect to $P(Y|G)$ as our training loss. Following the literature, for LAD we evaluate in a strict zero-shot setting, meaning that the model is only evaluated on unseen classes. For AwA2, we evaluate in a generalized zero-shot setting, meaning that the model is evaluated on both seen and unseen classes, so we mix the unseen classes with estimated labels and seen classes with given labels during training. We compare with three recent transductive, embedding-based ZSL methods: QFSL (Song et al., 2018), VCL (Wan et al., 2019), and WDVSc (Wan et al., 2019). For context, we also report results from three

standard inductive methods, ConSE (Norouzi et al., 2013), ESZSL (Romera-Paredes and Torr, 2015), SynC (Changpinyo et al., 2016), although they are at a disadvantage because they do not access the unlabeled data nor any information about the unseen classes. For LAD, we replicate and report the results of WDVSc and VCL using the same features.

Results We report the average results and 95% confidence intervals based on five random seeds in Table 2.2. Similar to the text classification tasks, NPLM significantly outperforms NC (an average of 9.8 percentage points on LAD and 21.5 percentage points on AwA2), and the ablations show that the end model generalizes beyond the PLFs. NPLM is also competitive with WDVSc, the top-performing ZSL method, either slightly underperforming or outperforming.

2.5 Conclusion

In this chapter, I addressed the limitation of limited expressiveness of labeling functions in conventional weak supervision frameworks, which restrict experts to single-class outputs and force them to simplify their nuanced knowledge into rigid rules. I introduced a novel probabilistic generative model that allows noisy labeling functions to output subsets of possible class labels, thereby capturing the natural complexity of domain expertise more effectively. Our empirical evaluations demonstrate that this approach improves the performance of weak supervision on text classification tasks and enables pre-trained attribute detectors to perform competitively in transductive zero-shot learning for object classification. By expanding the expressiveness of labeling functions, we reduce the need for exhaustive rule creation and empower domain experts to convey their knowledge more flexibly. This advancement lays a robust foundation for more efficient and scalable weak supervision systems towards the goal of enhancing expert knowledge integration in machine learning models.

CHAPTER 3

Learning to Compose Soft Prompts for Compositional Zero-Shot Learning

3.1 Introduction

Compositionality is the long-standing goal of artificial intelligence of creating new concepts by combining existing primitive concepts (Chomsky, 1956; Fodor and Pylyshyn, 1988; Hupkes et al., 2020; Lake and Baroni, 2018; Marcus, 2003). The practical advantage of compositionality for deep neural networks lies in the ability to build new classifiers by combining existing classifiers. In this work, we consider compositional zero-shot learning, a classification task where the model learns to predict unseen or novel compositions of primitive concepts (Naeem et al., 2021; Nagarajan and Grauman, 2018; Purushwalkam et al., 2019). Research on compositional zero-shot learning in language and vision focuses on attribute-object compositions such as `old tiger` and `young tiger`, where `tiger` is the object category described by the attributes `old` and `young`.

Existing methods for compositional zero-shot learning typically map attributes and objects to pretrained word embeddings and use a pretrained image encoder backbone to jointly align the image and the attribute-object text representations to learn compositionality (Li

et al., 2020b; Mancini et al., 2021a,b; Misra et al., 2017; Naeem et al., 2021; Nagarajan and Grauman, 2018; Purushwalkam et al., 2019; Xu et al., 2021). However, the pretraining of the word embeddings and image encoder is disjoint and isolated from each other, i.e., these methods learn to align image and text representations from scratch. These task-specific architectures also are limited in flexibility. For example, to adapt these methods to higher-order compositions with multiple attributes and objects such as `small furry cat` or `old white tiger`, the original architecture needs to be modified. The ability to generalize beyond the original training length is a key test for compositionality (Hupkes et al., 2020).

In contrast, we propose to build on large-scale pretrained vision-language models (VLMs), which are trained on massive amounts of aligned images and text (Jain et al., 2021; Jia et al., 2021; Li et al., 2021; Radford et al., 2021b). We focus on CLIP (Radford et al., 2021b), a powerful vision-language model pretrained on 400 million image-text pairs. CLIP has two main components: the image encoder and the text encoder that produce vector representations for images and text in a multi-modal embedding space. The text encoder accepts a textual input, or a prompt such as `A photo of dog` to produce a vector representation for the class `dog`. Taking the cosine similarity with all the class prompts and the image, we get a compatibility score for the classes and pick the one with the highest score. However, CLIP without any fine-tuning underperforms task-specific architectures, even though it has been pre-trained on vastly more data. This finding suggests that there is significant room for improvement from teaching VLMs like CLIP about composing concepts.

To improve VLMs for compositional zero-shot learning, we introduce compositional soft prompting ($\mathbb{CSP}$), a parameter-efficient learning technique that tunes tokens of vocabulary to represent primitive concepts in a composable way. Fine-tuning large pre-trained models such as CLIP requires huge amounts of compute and may lead to overfitting (Sung et al., 2021; Mitchell et al., 2022) (see also Section 3.5). This challenge has motivated

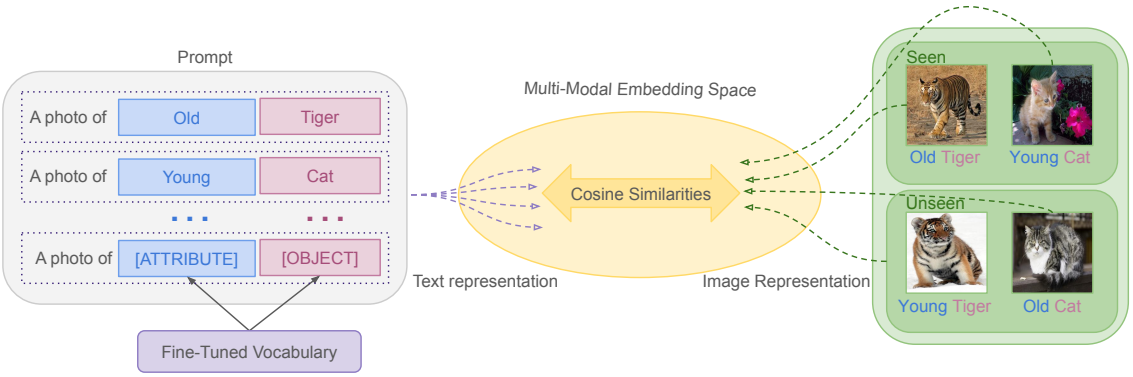

Figure 3.1: An overview of compositional zero-shot learning with ℂ𝕊ℙ. We fine-tune the vocabulary for attributes and objects on the seen classes. Then we compose novel soft prompts to test on the unseen classes.

several soft prompting techniques in both language and vision (Lester et al., 2021; Qin and Eisner, 2021; Vu et al., 2021; Zhou et al., 2021). These works tune a single prompt on a downstream supervised task, often in a few-shot setting. For instance, they typically use prompts such as `A photo of [class]` and tune the prefix `A photo of` on the entire dataset. In contrast, ℂ𝕊ℙ is a novel way of soft prompting. We treat the attributes and objects that are composed to define classes as learnable tokens of vocabulary in a prompt as `A photo of [attribute] [object]`. We tune on multiple `[attribute]` and `[object]` prompt compositions, and then we recompose them into new prompts for zero-shot inference (Figure 3.1).

Our results show that ℂ𝕊ℙ improves over the zero-shot performance of CLIP. ℂ𝕊ℙ significantly improves over CLIP across three benchmark datasets by an average accuracy of 13.7 percentage points in the closed-world setting and 8.0 percentage points in the open-world setting (using the AUC metric). ℂ𝕊ℙ also outperforms CoOp, a soft prompting method that tunes the prefix context, by an average of 7.3 percentage points in the closed-world setting and 4.3 percentage points in the open-world setting on the AUC metric.

In addition to improved benchmark accuracy, ℂ𝕊ℙ has several other advantages when tested on other kinds of zero-shot inferences without any changes to training. We show that the learned attribute vocabulary can be decomposed to better classify attributes

in isolation, using prompts of the form `A photo of [attribute] object`. We also show that training ℂ𝕊ℙ with attribute-object compositions improves CLIP's performance on attribute-attribute-object compositions. Finally, we show that ℂ𝕊ℙ improves generalization to compositions of *unseen* attributes and seen objects. Prior work on compositional zero-shot learning typically only evaluates unseen compositions of seen attributes and seen objects.

In summary, our main contributions are:

- We introduce compositional soft prompting (ℂ𝕊ℙ), a parameter-efficient learning technique to improve the compositionality of large-scale vision-language models (VLMs). The attributes and objects that are composed to define classes are treated as learnable tokens of vocabulary. Unlike existing work on soft prompting, our learned prompts are tuned on multiple compositions and then recomposed in new combinations for inference.
- ℂ𝕊ℙ improves the AUC accuracy of CLIP by an average of 10.9 percentage points across three benchmark datasets (Isola et al., 2015; Yu and Grauman, 2014; Mancini et al., 2021a). It also improves over CoOp, a soft-prompting method that tunes the prefix context, by an average of 5.8 percentage points on AUC.
- We conduct additional experiments to analyze ℂ𝕊ℙ. We show that training on attribute-object compositions improves CLIP's accuracy on attribute classification alone, attribute-attribute-object compositions, and compositions of pretrained and fine-tuned vocabulary.

3.2 Related Work

We describe the related work in prompting, parameter-efficient learning, and compositional zero-shot learning.

Prompting Prompting is a recent focus in the language and vision communities that has shown benefits in zero-shot and few-shot learning on a wide range of tasks (Bach et al., 2022a; Bommasani et al., 2021; **?**; Lester et al., 2021; Radford et al., 2021b; Qin and Eisner, 2021; Sanh et al., 2022; Vu et al., 2021; Zhou et al., 2021, 2022). Discrete prompts are typically hand-written text inputs that provide guidelines to large pre-trained models such as CLIP, GPT-3 (**?**), etc. for inference without updating the model parameters. While manually engineering prompts can help achieve better accuracy, it is often time-consuming and impractical to find the best prompt.

Soft prompting is an alternative to discrete prompts, where a part of the prompt is learned by backpropagating without fine-tuning the entire model. Several works using soft prompts show improved accuracy compared to hand-crafted prompts (Lester et al., 2021; Li and Liang, 2021; Qin and Eisner, 2021; Shin et al., 2020; Vu et al., 2021; Zhou et al., 2021). In all these works, soft prompts are a single input concatenated to all inputs for the entire task. In contrast, we learn tokens for each primitive concept from multiple compositions and recompose them in new ways to represent unseen classes for zero-shot inference. We show in Section 3.5 that traditional soft prompting can also improve CLIP on compositional zero-shot learning, but generally not as much as $\mathbb{CSP}$.

Recent works have used soft prompting for large-scale vision-language models. Zhou et al. (2021) propose CoOp (contextual optimization), a soft prompting method for few-shot object classification. Other applications include visual question answering (Jin et al., 2022) and video understanding (Ju et al., 2021). Again, these works tune a single soft prompt on the entire dataset. One exception is Ge et al. (2022), which learns multiple soft prompts for cross-domain adaption. While our work shares similarities with their work, there are important differences. Our work decomposes the class labels into multiple parts rather than splitting the prompt into domain-related granularities such as domain-agnostic context, domain-specific context, and class label. Furthermore, we focus on compositional zero-shot learning where we do not have access to labeled examples from the unseen

classes in the test set whereas they assume access to all the test classes during training.

Zhou et al. (2022) extend CoOp to CoCoOp (conditional contextual optimization) to reduce overfitting in prompt tuning for few-shot object classification. They learn a lightweight network that takes the image representation and produces a conditional vector that is added to the soft tokens in the prompt.

Parameter-Efficient Learning Soft prompting is closely related to the growing body of work in parameter-efficient learning (Houlsby et al., 2019; Guo et al., 2021; Mahabadi et al., 2021; Sung et al., 2021; Ben-Zaken et al., 2022; Liu et al., 2022). They add small feedforward networks between the layers in the pretrained model, or use sophisticated techniques to select a sparse set of model parameters and update them by fine-tuning on a labeled training set. However, unlike $\mathbb{CSP}$, the methods do not assign semantic meaning to the parameters in order to enable composition.

Compositional Zero-Shot Learning The growing interest in compositional zero-shot learning has contributed to several architectural innovations (Li et al., 2020b; Mancini et al., 2021a,b; Misra et al., 2017; Naeem et al., 2021; Nagarajan and Grauman, 2018; Purushwalkam et al., 2019; Radenović et al., 2021). Early works compose attributes and objects with a transformation function (Misra et al., 2017; Nagarajan and Grauman, 2018). Recent work uses separate encoding layers for attributes and objects, and then combines them with late fusion using a linear layer or a multilayer perceptron (Purushwalkam et al., 2019). The most successful methods represent the attribute and object relationship in a graph and learn their compositions via graph convolutional networks (Mancini et al., 2021b; Naeem et al., 2021; Ruis et al., 2021). Yun et al. (2022) study the emergence of compositionality in CLIP pre-training by learning linear classifiers rather than soft prompts for primitive concepts. Scialom et al. (2022) show that continually fine-tuning large language models can learn composable instructions.

Evaluating CLIP in a fair setting compared to the existing task-specific architectures for

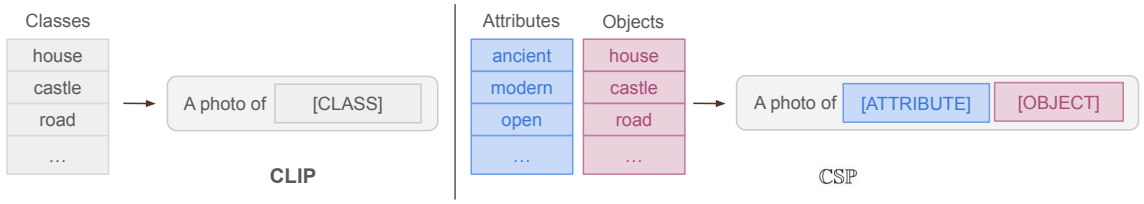

Figure 3.2: Comparison of CLIP in a zero-shot and $\mathbb{CSP}$ in a compositional zero-shot setting.

compositional zero-shot learning is challenging. On the one hand, CLIP is trained on a web-scale dataset to which other methods do not have access, and may even contain some of the classes from the unseen split. On the other hand, existing task-specific architectures fine-tune orders of magnitude more parameters than soft prompting, while using specialized architectures that do not adapt as easily as VLMs to new tasks. For these reasons, our work focuses on improving CLIP-based baselines, and we include a summary of the results comparing task-specific architectures in Section 3.5.

Compositional zero-shot learning is also closely related to the broader goal of compositionality in artificial intelligence (Chomsky, 1956; Fodor and Pylyshyn, 1988). Existing works have studied compositionality for image generation (Herzig et al., 2020), video synthesis (Ye et al., 2019; Bar et al., 2021; Nawhal et al., 2020), visual reasoning (Johnson et al., 2017), semantic parsing (Drozdov et al., 2022), language grounding (Jin et al., 2022), and question answering (Yuan et al., 2019). For more in-depth discussion on compositionality, we refer the readers to Hupkes et al. (2020).

3.3 Preliminaries

In this section, we formally define the task of compositional zero-shot learning and describe how to use CLIP for compositional zero-shot inference. Let $\mathbb{A} = \{a_0, a_1, ..., a_n\}$ be the set of possible attributes and $\mathbb{O} = \{o_0, o_1, ..., o_m\}$ be the set of possible object categories. Let the label space $\mathbb{Y}$ be the Cartesian product of the attribute set and the object category set, $\mathbb{Y} = \mathbb{A} \times \mathbb{O}$. We are given two disjoint label subsets such that $\mathbb{Y}_{seen} \subset \mathbb{Y}$, $\mathbb{Y}_{unseen} \subset \mathbb{Y}$, and $\mathbb{Y}_{seen} \cap \mathbb{Y}_{unseen} = \varnothing$ where $\mathbb{Y}_{seen}$ and $\mathbb{Y}_{unseen}$ are the set of the seen and unseen

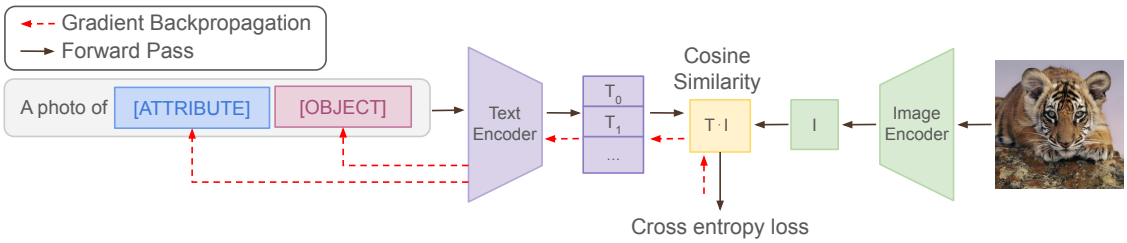

Figure 3.3: Training setup for $\mathbb{CSP}$. The prompt with the attribute and object vocabulary is passed through the text encoder to get the text representation. The example is passed through the image encoder for the image representation. Next, we take the cosine similarity for all the prompts with the image and compute the cross entropy loss. Finally, we backpropagate the loss through the text encoder and update the attribute and object vocabulary weights.

classes. At training time, we are given examples $\mathbb{S}_{seen} = \{(x_1, y_1), ..., (x_n, y_n)\}$ to learn some discriminative model $f : \mathbb{X} \rightarrow \mathbb{Y}_{seen}$. During inference, we want the model to predict both seen and unseen classes in the test set, i.e., $f : \mathbb{X} \rightarrow \mathbb{Y}_{test}$. In the closed-world evaluation, the test set is defined as $\mathbb{Y}_{test} = \mathbb{Y}_{seen} \cup \mathbb{Y}_{unseen}$. In the open-world evaluation, the model has to consider all possible permutations of the attribute-object compositions, i.e., $\mathbb{Y}_{test} = \mathbb{Y}$ and $\mathbb{Y}_{unseen} = \mathbb{Y} - \mathbb{Y}_{seen}$.

We can easily adapt CLIP to compositional zero-shot inference by defining prompts appropriately. The prompts used in CLIP for zero-shot inference differ from the prompts used in works like GPT-3 (**?**) and T0 (Sanh et al., 2022). Instead of defining one prompt for an N-way classification task, the N classes are transformed into natural language prompts such as `A photo of [class]` (Figure 3.2, left). This results in N prompt vectors, which are used to compute the cosine similarity with the image representation for prediction. For compositional zero-shot inference, we replace the `[class]` placeholder with the attribute and object composition as follows: `a photo of [attribute] [object]`. The change in prompt format allows us to represent pairs of attribute and objects such as `a photo of young cat` in the text encoder without changes.

3.4 Compositional Soft Prompting

In this section, we introduce compositional soft prompting ($\mathbb{CSP}$), a parameter-efficient learning technique for fine-tuning large pretrained models for better compositionality.

Motivation The goal of our work is to improve VLMs such as CLIP on compositional generalization where they seem to underperform the current state-of-the-art methods This is perhaps because CLIP's pretraining on data crawled from the web does not provide sufficient supervision about attributes and how they combine with different objects. Therefore, we aim to teach VLMs such as CLIP how to better compose primitive concepts. We approach this is a vocabulary-learning problem because it is parameter efficient and gives us a natural way to compose new classes.

Prompt Construction $\mathbb{CSP}$ treats the attributes and objects that are composed to define classes as learnable tokens of vocabulary and tunes them on multiple prompt compositions. We represent each primitive concept, either attribute or object, as a new, auxiliary token in the VLM's vocabulary. To represent a class, we combine a fixed prefix and the corresponding primitive concepts, for example `A photo of young tiger`, where `young` maps to the auxiliary weight vector for the corresponding attribute and `tiger` maps to the auxiliary weight vector for the corresponding object. Then, we use these prompts for compositional zero-shot learning in the same way as we would for CLIP, as described in Section 3.3. Training and inference with $\mathbb{CSP}$ is very simple as we only need to swap the vocabulary of the attributes and objects for any desired composition in the prompt (Figure 3.2, right). As a result, our method tunes only $(|\mathbb{A}| + |\mathbb{O}|) \times d$ parameters where d is the dimension of the vocabulary embedding. This is a novel form of soft prompting, because prior work (Lester et al., 2021; Qin and Eisner, 2021; Zhou et al., 2021) tune prompts for seen classes and usually one prompt for the whole task, whereas we compose learned prompt tokens to represent unseen classes at test time.

Training We want to learn vector embeddings for the new, auxiliary vocabulary:

$\boldsymbol{\theta} = [\boldsymbol{\theta}_{\mathbb{A}}; \boldsymbol{\theta}_{\mathbb{O}}]$ where $\boldsymbol{\theta} \in \mathbb{R}^{(|\mathbb{A}|+|\mathbb{O}|)\times d}$. We learn them by composing prompts with them and fine-tuning them on the seen classes. Figure 3.3 shows the overall learning process for $\mathbb{CSP}$.

First, we initialize the learnable vocabulary with pretrained embeddings from CLIP using the concept names, i.e. attributes and objects. If a concept, such as `Faux Fur`, has multiple tokens, we average their pre-trained representations to initialize the learnable vocabulary. Next, for each class, we construct a prompt with the pretrained vocabulary for the prefix context and learnable vocabulary for the primitive concepts:

$$t_{a,o} = (\boldsymbol{w}_0, \ldots, \boldsymbol{w}_p, \boldsymbol{\theta}_a, \boldsymbol{\theta}_o)$$

where $\boldsymbol{w}_i \in \mathbb{R}^d$ are the tokens of the prefix context, and $\boldsymbol{\theta}_a$ and $\boldsymbol{\theta}_o$ are the learnable parameters for the attribute and the object in the prompt.

Next, we pass the prompt with learnable parameters through the text encoder to get the text representation:

$$\boldsymbol{t}_{a,o} = \frac{VLM_{\mathbf{T}}\ (t_{a,o})}{||\, VLM_{\mathbf{T}}\ (t_{a,o})||}$$

Then, for some image x, we get the image representation from the image encoder:

$$\boldsymbol{x} = \frac{VLM_{\mathbf{V}}\ (x)}{||\, VLM_{\mathbf{V}}\ (x)||}$$

Finally, we compute the class probability:

$$p_{\boldsymbol{\theta}}(y = (a, o) \mid x) = \frac{\exp\left(\boldsymbol{x} \cdot \boldsymbol{t}_{a,o}/\tau\right)}{\sum_{(\hat{a},\hat{o})\in \mathbb{Y}_{seen}} \exp\left(\boldsymbol{x} \cdot \boldsymbol{t}_{\hat{a},\hat{o}}/\tau\right)}$$

where $\tau \in \mathbb{R}$ is the temperature parameter from CLIP. We learn the parameters $\boldsymbol{\theta}$ by

	Composition			*Train*		*Validation*			*Test*		
Dataset	$\lvert\mathbb{A}\rvert$	$\lvert\mathbb{O}\rvert$	$\lvert\mathbb{A}\times\mathbb{O}\rvert$	$\lvert\mathbb{Y}_{seen}\rvert$	$\lvert\mathbb{X}\rvert$	$\lvert\mathbb{Y}_{seen}\rvert$	$\lvert\mathbb{Y}_{unseen}\rvert$	$\lvert\mathbb{X}\rvert$	$\lvert\mathbb{Y}_{seen}\rvert$	$\lvert\mathbb{Y}_{unseen}\rvert$	$\lvert\mathbb{X}\rvert$
MIT-States Isola et al. (2015)	115	245	28175	1262	30338	300	300	10420	400	400	12995
UT-Zappos Yu and Grauman (2014)	16	12	192	83	22998	15	15	3214	18	18	2914
C-GQA Mancini et al. (2021a)	413	674	278362	5592	26920	1252	1040	7280	888	923	5098

Table 3.1: Summary statistics of the datasets used in our experiments.

minimizing the cross entropy loss on the training dataset:

$$-\frac{1}{|\mathbb{S}_{seen}|}\sum_{(x,y)\in\mathbb{S}_{seen}}\log p_{\boldsymbol{\theta}}(y \mid x)+\lambda||\boldsymbol{\theta}||^2$$

where $\lambda \in \mathbb{R}$ is the weight decay.

Inference During inference, we recompose the fine-tuned attribute and object vocabulary in the prompt. We compose the candidate prompts with the tuned $\boldsymbol{\theta}$ with the (attribute, object) pairs in the same way during training. In both closed-world and open-world settings, we only replace attribute and objects with the fine-tuned parameters in the prompt. Finally, we calculate the most likely attribute and object pair as follows:

$$\hat{y}=\underset{y\in\mathbb{Y}_{test}}{\operatorname{argmax}}\ p_{\boldsymbol{\theta}}(y \mid x)$$

3.4.1 Pseudocode

Figure 3.4 shows the Torch-like pseudocode for inference with $\mathbb{CSP}$. The function accepts the minibatch of images, test pairs, and the clip model with the fine-tuned embeddings and returns the cosine similarities between the image representation and the text representation scaled by a constant scalar.

3.5 Experimental Evaluation

In this section, we describe our experiments with $\mathbb{CSP}$. We compare $\mathbb{CSP}$ to CLIP-based baselines in the closed-world and open-world settings of compositional zero-shot learning. We also compare the performance of fine-tuned CLIP with $\mathbb{CSP}$on the benchmark datasets. Finally, we demonstrate that $\mathbb{CSP}$ can generalize beyond these benchmarks to three

```
def inference(
    batch_images: nn.Tensor,
    test_pairs: List[List, List],
    model: nn.Module
):
    """
    Function to run inference with the fine-tuned embeddings.
    Args:
        batch_images (torch.Tensor): minibatch of images [n, h, w, c]
        test_pairs (tuple): attribute-object pairs in the test
            split [m, 2]
        model (nn.Module): model with the fine-tuned embeddings
    Returns:
        torch.Tensor: cosine similarties of the minibatch images
            and attribute-object pairs [n, m]
    """
    prompt_template = "a photo of x x"
    tokenized_prompt = tokenize(prompt_template)
    tokenized_prompt = tokenized_prompt.repeat(len(test_pairs))
    token_tensor = model.token_embedding(tokenized_prompt)

    # fine-tuned embeddings
    attr, obj = zip(*test_pairs)
    attr_emb = model.soft_embedding(attr)
    obj_emb = model.soft_embedding(obj)

    # replace the "x x" in prompt template with fine-tuned embeddings
    token_tensor = replace_emb(token_tensor, attr_emb, obj_emb)

    # l2-normalized
    text_rep = model.text_encoder(token_tensor)
    image_rep = model.image_encoder(batch_images)

    logits = (image_rep @ text_rep) * model.logit_scale.exp()

    return logits
```

Figure 3.4: Torch-like pseudocode for inference with $\mathbb{CSP}$.

modified settings: attribute-only classification, attribute-attribute-object composition, and inference with unseen attributes.

Dataset

We experiment with three attribute-object composition benchmarks: MIT-states (Isola et al., 2015), UT-Zappos (Yu and Grauman, 2014), and C-GQA (Naeem et al., 2021). Table 3.1 summarizes the statistics of the datasets. MIT-states contains images of naturally occurring objects where each object is described by an adjective. UT-Zappos contains images of shoes paired with fine-grained states. For this dataset, we use the split suggested by Purushwalkam et al. (2019). C-GQA, a newly introduced dataset derived from the Stanford GQA dataset (Hudson and Manning, 2019), contains images of objects paired with states.

		MIT-States				*UT-Zappos*				*C-GQA*			
	Method	S	U	H	AUC	S	U	H	AUC	S	U	H	AUC
Closed	CLIP	30.2	46.0	26.1	11.0	15.8	49.1	15.6	5.0	7.5	25.0	8.6	1.4
	CoOp	34.4 $_{0.1}$	47.6 $_{0.1}$	29.8 $_{0.1}$	13.5 $_{0.0}$	52.1 $_{0.5}$	49.3 $_{1.8}$	34.6 $_{1.7}$	18.8 $_{1.4}$	20.5 $_{0.2}$	**26.8** $_{0.3}$	17.1 $_{0.2}$	4.4 $_{0.1}$
	$\mathbb{CSP}$	**46.6** $_{0.1}$	**49.9** $_{0.1}$	**36.3** $_{0.1}$	**19.4** $_{0.1}$	**64.2** $_{0.7}$	**66.2** $_{1.2}$	**46.6** $_{1.2}$	**33.0** $_{1.3}$	**28.8** $_{0.1}$	**26.8** $_{0.1}$	**20.5** $_{0.1}$	**6.2** $_{0.0}$
Open	CLIP	30.1	14.3	12.8	3.0	15.7	20.6	11.2	2.2	7.5	4.6	4.0	0.27
	CoOp	34.6 $_{0.1}$	9.3 $_{0.0}$	12.3 $_{0.1}$	2.8 $_{0.0}$	52.1 $_{0.5}$	31.5 $_{2.9}$	28.9 $_{2.3}$	13.2 $_{1.6}$	21.0 $_{0.2}$	4.6 $_{0.1}$	5.5 $_{0.1}$	0.70 $_{0.0}$
	$\mathbb{CSP}$	**46.3** $_{0.3}$	**15.7** $_{0.1}$	**17.4** $_{0.1}$	**5.7** $_{0.0}$	**64.1** $_{0.7}$	**44.1** $_{0.3}$	**38.9** $_{0.5}$	**22.7** $_{0.4}$	**28.7** $_{0.2}$	**5.2** $_{0.1}$	**6.9** $_{0.1}$	**1.20** $_{0.0}$

Table 3.2: Closed-world (Closed) and open-world (Open) results on MIT-States, UT-Zappos, and C-GQA. For CoOp and $\mathbb{CSP}$, we report the average performance of the models on 5 random seeds with standard error.

Benchmark Evaluation We follow the standard closed-world and open-world evaluation protocols. In the closed-world setting, the unseen classes are a subset of all possible attribute-object combinations and are defined in the dataset. In the open-world setting, the model considers all possible attribute-object combinations.

Following prior work (Mancini et al., 2021a), we report the performance in the generalized zero-shot learning for both the closed-world and the open-world settings. In generalized zero-shot learning, we include both the seen and unseen classes in the test set. Several works have noted that zero-shot models are biased towards the seen classes, so we follow the standard of adding a scalar bias to the unseen classes (Chao et al., 2016; Ruis et al., 2021) . Following prior work, we vary the bias from $-\infty$ to $+\infty$ to get a curve indicating the seen accuracy on the x-axis and unseen accuracy on the y-axis. We report the area under the curve (AUC) and select the operating point with the best harmonic mean (H) between the seen and unseen accuracy. We also report the best seen accuracy (S) when bias is $-\infty$ and the best unseen accuracy (U) when the bias is $+\infty$. We average over five random seeds, selecting the best-performing model after each epoch on the validation data.

Following prior work, we apply feasibility calibration to all methods for prediction in the open-world setting. The open-world setting is particularly challenging as the label space contains all possible combinations of attributes and objects in the dataset. For instance, the label space contains feasible compositions such as `young cat` and `eroded cliff` and infeasible compositions such as `eroded cat`. Existing work shows a significant drop in

model performance from the closed-world setting to the open-world setting (Mancini et al., 2021a,b). As suggested in Mancini et al. (2021a), we apply a simple feasibility calibration based on GloVe embeddings (Pennington et al., 2014) to filter out infeasible compositions.

Baselines In our experiments, we primarily compare with CLIP-based baselines. We compare $\mathbb{CSP}$ to pretrained CLIP (Radford et al., 2021b) and CoOp (Zhou et al., 2021). CoOp is a soft-prompting method that learns the prefix context with limited labeled examples in a few-shot setting. CoOp resembles prompt tuning (Lester et al., 2021) applied to VLMs.

Training Details We implement $\mathbb{CSP}$ and the baselines with a pretrained CLIP model in PyTorch (Paszke et al., 2019). We use the CLIP model ViT-L/14 which is the largest available model in our experiments. Nonetheless, our method is agnostic to the choice of CLIP architecture. The pretrained CLIP ViT-L/14 model has a vision transformer (ViT) as the image encoder and a transformer as the text encoder.

We train $\mathbb{CSP}$ and CoOp by minimizing the cross entropy loss with the Adam optimizer over the seen split in the dataset for 20 epochs. We use a single NVIDIA RTX 3090 or V100 GPU depending on their availability to train all our models. For each dataset, we choose the best hyperparameters based on the performance on the validation split.

Benchmark Results Our results in Table 3.2 show that $\mathbb{CSP}$ significantly improves over CLIP on all the benchmark datasets in the closed-world and open-world settings. In the closed-world setting, we outperform CLIP in the AUC metric by 8.4 points on MIT-states, 28.0 point on UT-Zappos, and 4.8 points on C-GQA. Additionally, we show that $\mathbb{CSP}$ beats CoOp, a soft-prompting method, in the AUC metric by 5.9 points on MIT-States, 14.2 points on UT-Zappos, and 1.8 points on C-GQA. In results in the open-world setting show that $\mathbb{CSP}$ improves over CLIP by 2.7 points on MIT-States, 20.5 points on UT-Zappos, and 0.93 points on C-GQA in the AUC metric. We outperform CoOp on MIT-States by 3.1 points, UT-Zappos by 9.5 points, and C-GQA by 0.50 points

Method	*MIT-States*	*UT-Zappos*	*C-GQA*
CLIP	11.0	5.0	1.4
CLIP (FT)	**22.2** $_{0.1}$	24.5 $_{1.6}$	**10.5** $_{0.2}$
$\mathbb{CSP}$	19.4 $_{0.1}$	**33.0** $_{1.3}$	6.2 $_{0.0}$

Table 3.3: Closed-world results comparing $\mathbb{CSP}$ and fine-tuned CLIP (FT). For $\mathbb{CSP}$ and CLIP (FT), we report the average AUC on 5 random seeds with standard error.

Method	*MIT-States*	*UT-Zappos*	*C-GQA*
ProtoProp	-	**34.7**	-
CGE	6.5	33.5	4.2
Co-CGE	6.6	33.9	4.1
CLIP	11.0	5.0	1.4
$\mathbb{CSP}$	**19.4** $_{0.1}$	33.0 $_{1.3}$	**6.2** $_{0.0}$

Table 3.4: Closed-world results comparing $\mathbb{CSP}$ with task-specific architectures. For $\mathbb{CSP}$, we report the average AUC on 5 random seeds with standard error.

in the AUC metric.

Comparison with Full Fine-Tuning

We aim to understand the potential benefits of fine-tuning all the parameters of CLIP instead of a small number. To that end, we fine-tune all the parameters in CLIP on the seen classes in our benchmark datasets.

Table 3.3 shows that fine-tuning CLIP can improve generalization to unseen classes but still achieves lower AUC compared to $\mathbb{CSP}$ on UT-Zappos. In addition, fine-tuning CLIP requires GPUs with more memory and often meticulous hyperparameter selection (Dong et al., 2022).

Comparison with Task-Specific Architectures

We compare $\mathbb{CSP}$ to existing task-specific compositional zero-shot learning architectures. We consider the following task-specific architectures: CGE (Naeem et al., 2021), Co-CGE (Mancini et al., 2021b) and ProtoProp (Ruis et al., 2021). These methods are the most recent best-performing methods in the closed-world setting.

Our results in Table 3.4 show that ℂ𝕊ℙ outperform existing task-specific architectures on MIT-States and C-GQA while being competitive on UT-Zappos in AUC. We see that ℂ𝕊ℙ improves over existing task-specific architectures by 12.8 points on MIT-states and 2.0 points on C-GQA in AUC.

Method	Accuracy
CLIP	62.7
CoOp	65.2 $_{0.3}$
ℂ𝕊ℙ	**72.6** $_{0.4}$

Table 3.5: Unseen accuracy on 5 random seeds with std. error.

Generalization to Higher-Order Compositions To test the additional flexibility afforded by VLMs, we test if training ℂ𝕊ℙ with attribute-object compositions can generalize to higher-order compositions such as attribute-attribute-object compositions.

We annotate a novel challenge dataset: AAO-MIT-States, a subset derived from the MIT-States dataset. In this dataset, we annotate the images in the test split of the MIT-States dataset with an additional attribute, to get an attribute-attribute-object pair as the class label.

We compare CLIP, CoOp, and ℂ𝕊ℙ to classify images with attribute-attribute-object classes. Since these class compositions are not present during training, we treat them as unseen classes and calculate the unseen accuracy. We take the best performing models for MIT-States and run inference on the challenge dataset.

Table 3.5 shows that ℂ𝕊ℙ improves over CLIP by an average 9.9 percentage points on unseen accuracy and generalizes to attribute-attribute-object compositions without any modifications or training. The results demonstrate that ℂ𝕊ℙ improves the compositionality of CLIP's vocabulary, even in ways that were not explicitly supervised. **Generalization to Mixed Vocabulary**

To further test the additional flexibility afforded by VLMs, we also evaluate ℂ𝕊ℙ on

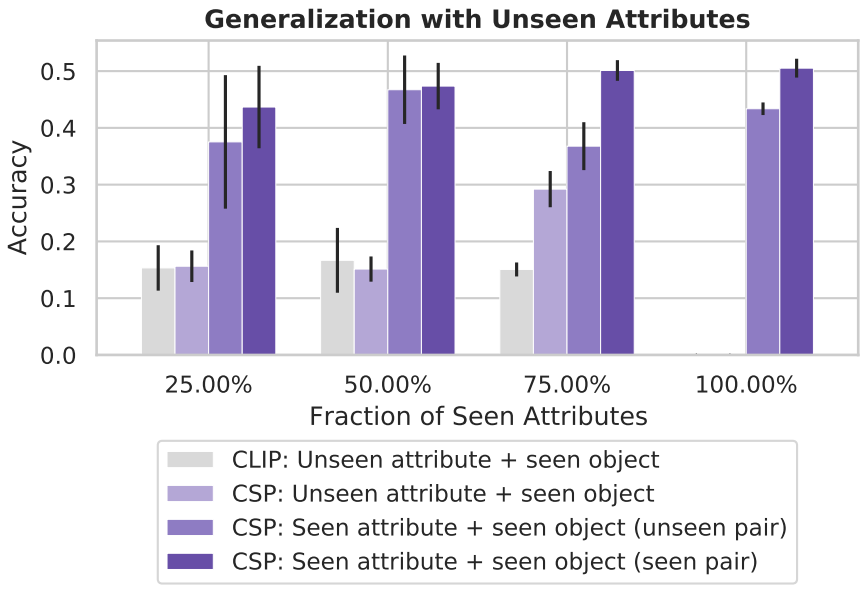

Figure 3.5: Results of ℂ𝕊ℙ and CLIP with different fractions of pretrained and fine-tuned vocabulary. In each fraction, we report the average performance of CLIP and ℂ𝕊ℙ on 5 random attribute splits.

compositional zero-shot learning with a mixture of pretrained and fine-tuned vocabulary. This evaluation stems from the practical need to combine new unseen attributes with fine-tuned vocabulary. Evaluating in this setting will allow us to assess whether the benefits of ℂ𝕊ℙ extend to classes including vocabulary not seen during fine-tuning. This setup goes beyond the above benchmarks, which include unseen *combinations* of attributes and objects, but all attributes and objects are seen during training. Now, we include completely unseen attributes.

We apply ℂ𝕊ℙ on UT-Zappos with different fractions of attributes as seen attributes. We randomly select 25%, 50%, 75%, and 100% of the attributes and all the objects from the training set. Then, we remove from the seen classes the attribute-object pairs that include an unseen attribute. Finally, we train the on the remaining seen attribute-object pairs with five random seed values.

For each split of the seen and unseen attributes, we evaluate CSP by dividing the classes into three buckets: (1) unseen attribute + seen object pairs, (2) seen (i.e., fine-tuned)

attribute + seen object pairs in unseen combinations, and (3) seen attribute + seen object pairs in seen combinations. In this evaluation, we refer to the classes in the first and second buckets as the unseen classes and those in the third bucket as the seen classes. This evaluation is more general than typical compositional zero-shot learning, which only evaluates on classes in the second and third buckets. Similar to our evaluation in Section 3.5, we add a scalar bias to the unseen classes and select the bias that maximizes the harmonic mean between accuracy on the unseen and seen classes in the validation set of UT-Zappos. We then report accuracy on unseen examples in each of the three buckets. To contextualize the performance of $\mathbb{CSP}$, we report the accuracy of CLIP on the unseen attribute + seen object pairs.

Figure 3.5 shows that the performance on unseen attribute + seen object pairs improves with $\mathbb{CSP}$ and sufficient training pairs. Initially, the performance of CLIP and $\mathbb{CSP}$ are comparable but by providing more combinations of supervision for the objects $\mathbb{CSP}$ significantly outperforms CLIP on the unseen attribute + seen object evaluation bucket. These results demonstrate that the fine-tuned vocabulary of $\mathbb{CSP}$ can improve the compositional zero-shot performance of pretrained vocabulary.

3.6 Conclusion

In this chapter, I addressed the limitation of existing models' inability to handle compositional knowledge—specifically, their struggle to recognize novel combinations of known concepts without requiring separate annotations for each. I introduced $\mathbb{CSP}$, a compositional soft prompting technique designed to overcome the limited capability of existing models to handle compositional knowledge. By treating attributes and objects as learnable vocabulary tokens within vision-language models like CLIP, our method enables the recognition of novel attribute-object combinations without the need for separate annotations for each possible pair. This approach directly addresses the scalability issue, where the exponential growth of potential combinations makes comprehensive labeling impractical.

Our experiments demonstrate that $\mathbb{CSP}$ significantly enhances compositional zero-shot performance while maintaining computational efficiency with only a small number of additional parameters. The learned vocabulary not only generalizes to unseen combinations but also adapts to more complex, higher-order compositions. By empowering models to compose new concepts from existing ones, $\mathbb{CSP}$ reduces the reliance on extensive labeled datasets and facilitates more efficient knowledge transfer from domain experts.

CHAPTER 4

Alfred: A System for Prompted Weak Supervision

[1]

4.1 Introduction

Acquiring labeled data is a significant challenge for machine learning for its time-consuming and expensive nature. Programmatic weak supervision (PWS) provides a more efficient method of data annotation by using noisy heuristics to label data. In a typical PWS setup, domain experts design labeling functions (LFs) as programs that vote for a label or abstain. (Ratner et al., 2016b, 2017) Recently, there has been a growing interest in creating LFs from large, pre-trained models through prompting (Smith et al., 2022; Arora et al., 2022; Zhang et al., 2022b). In the shift to this new setting, executing LFs goes from the least to the most computationally expensive part of the process, highlighting the importance of providing a software infrastructure that facilitates efficient development. However, existing toolkits for large language models mainly prioritize prompt templating

[1] https://github.com/BatsResearch/alfred

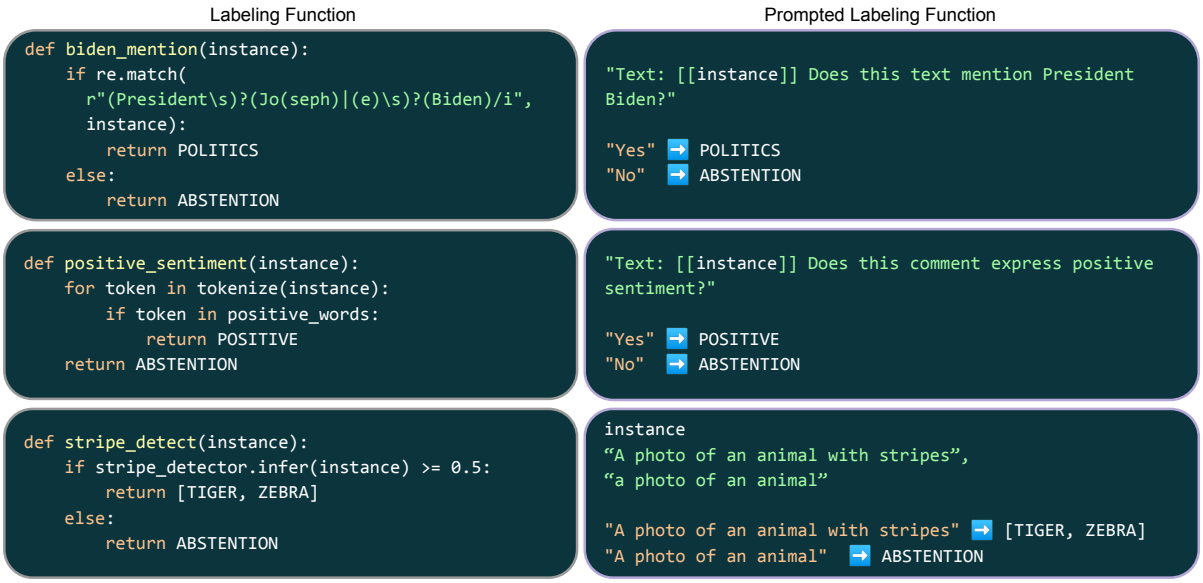

Figure 4.1: Examples of a labeling function versus a prompted labeling function. For the first example, each expresses supervision relating mentions of President Biden to the category of politics. Instead of specifying an intricate regular expression, a prompted labeling function uses the prompt “Text: [[instance]] Does this text mention President Biden?” where [[instance]] is replaced by the news article to be labeled. The response is mapped to a vote on the true label. The second example demonstrates how heuristics about positive sentiment that were previously hard to define can be flexibly expressed as a natural language question. Instead of defining a set of keywords for fuzzy sentiment matching, we can simply ask large pretrained models for answers about the sentiment. For the third example, we consider an animal labeling task where we use the visual attributes “stripes” to vote for the classes TIGER and ZEBRA. Previously, we would need to first collect supervised training data for attributes like stripes and then train classifiers to make the decisions. With modern vision-language models (e.g. CLIP), we can simply express the attribute detection task as a set of candidate prompts.

and tuning, leaving an unmet need for a system that that connects prompting with the creation of training data.

Prompted models offer a unique opportunity to enhance existing PWS systems. Traditional PWS systems require programming LFs with code that specifies heuristic domain knowledge. With large pre-trained models, natural language-based prompts can be used as LFs, also known as prompted LFs (Smith et al., 2022). This approach allows the easy expression of complex rules that were previously difficult to specify using code, as the example in Figure 4.1 shows. The ability to use prompts to define labeling functions simplifies and streamlines the weak supervision process, as well as potentially elevating

the quality of the annotations. This benefit is particularly helpful for tasks involving computer vision, where previously PWS has been limited to tasks for which models can identify key features (such as objects) over which to program rules. Whether the domain is language or multi-modal, prompts let users experiment with different heuristics (and phrasings of those heuristics). Therefore, enabling an iterative development experience is essential for the success of a prompt-based PWS system.

The switch to prompted models for weak supervision also presents significant challenges. It first requires rethinking the abstractions and workflow of first-generation PWS systems. Instead of editing code and managing libraries of functions, users must manage libraries of prompts, track their outputs on multiple datasets, and develop strategies for mapping those outputs to labels. This change is complicated by large models' demand for computational resources. The throughput of the models is a new development bottleneck. The extra overhead of remotely hosted models is a further hindrance to the iterative workflow of weak supervision.

Existing open-source software for prompting concentrates on prompt engineering (Orr, 2022; Bach et al., 2022b), prompt chains (Chase, 2022) or continuous (i.e., soft) prompt tuning (Ding et al., 2022); placing less emphasis on throughput for a large-model-in-the-loop workflow. Additionally, many existing open-source systems have not developed support for vision-language models, despite their benefits for data labeling. Incorporating large pre-trained models into a PWS system remains an open problem in the open-source software space, requiring innovative solutions to address these challenges and complement existing software focused on other aspects of prompting.

We present Alfred, a versatile PWS system that leverages large pre-trained models for image and text annotations. Alfred aims to provide an environment for the rapid development of prompt-based supervision, while maintaining a consistent development experience similar to established PWS systems. We designed Alfred with usability and efficiency in mind, aiming to provide a rapid and smooth experience for developing prompt-based supervision.

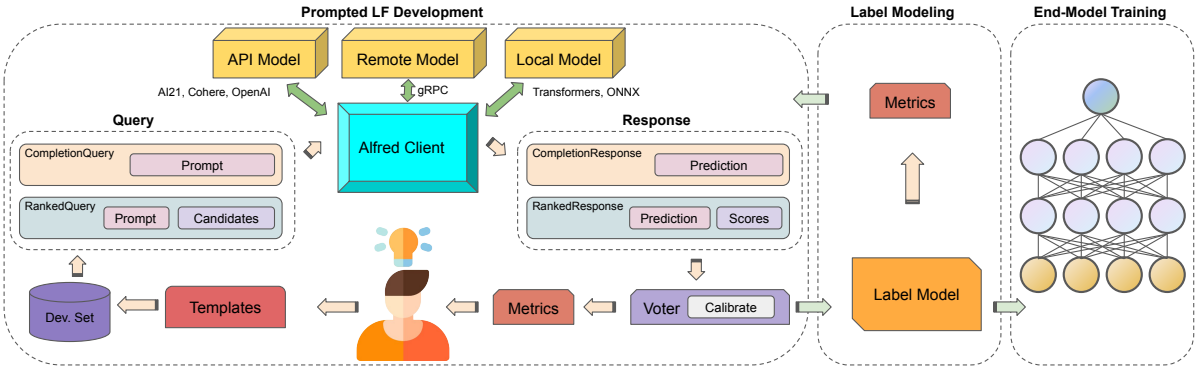

Figure 4.2: A typical workflow for programmatic weak supervision with Alfred. First, developers use Alfred to iteratively design, evaluate, and refine their prompted labeling functions (LFs). They use prompt Templates to generate Queries to Models based on data. The responses of Models are mapped to votes on the true labels for examples by Voters, which can be calibrated. Then the included Label Models combine the noisy votes to produce probabilistic training labels for an end model.

Alfred supports popular large language models from Hugging Face's transformer package (Wolf et al., 2020), including the GPT family (Radford et al., 2019), the T5 family (Raffel et al., 2020), etc., and vision-language models like CLIP (Radford et al., 2021a), etc. Alfred also supports local ONNX models, or API-based models from OpenAI, AI21, and Cohere. Moreover, Alfred provides easy templating tools to help users quickly create, evaluate, and refine prompted LFs. Alfred offers easy ways to deploy inference servers remotely, in addition to local model hosting. Alfred also optimizes model inference throughput with improved batching techniques and provides utilities for efficient LLM deployment and interaction. Finally, Alfred contains a library of label models to distill the outputs of prompted labeling functions into the final training datasets for downstream end models.

Alfred is a prototype for a second generation of PWS systems with prompting at their core. To this end, we highlight three key feature of Alfred:

- **Prompt-based weak supervision for images and text.** Alfred provides the necessary tools for users to create, evaluate, and refine prompt templates for both image and text weak supervision tasks. The inclusion of a query caching system that automatically stores and updates model responses facilitates development.

- **Optimized inference throughput.** Alfred implements a dynamic batching mechanism that optimizes large sets of prompts. This feature allows models hosted by Alfred to achieve 2-3× greater throughput than naive implementations.
- **Seamless local development experience.** Alfred can host models remotely and make them accessible to developers via gRPC, a high-throughput protocol for remote procedure calls.[2] It enables sending datasets to servers in large chunks to maintain higher throughput. Alfred also implements a SSH-based port-forwarding utility for the gRPC connection, easing deployment on shared clusters such as those at universities.

4.2 Related Work and Background

Alfred sits at the intersection of programmatic weak supervision and large pretrained models. In this section, we overview the most related work.

Programmatic Weak Supervision. Traditionally, supervised learning relied on manually labeled data, and data labeling has been seen as a key bottleneck for many applications. Recently, a family of programmatic weak supervision (PWS) methods have offered an alternative to costly manual annotations by incorporating multiple sources of noisy labels to create training datasets(Zhang et al., 2022a). Typically, a PWS system such as Snorkel (Ratner et al., 2017) has a three-stage setup: First, developers will create heuristics called labeling functions (LFs) that vote on the label for an example (or abstain). Second, a label model will reconcile the noisy votes and provide probabilistic labels for the data. Finally, freshly annotated data is used to train an end model with a noise-aware loss objective (e.g. in a classification setting, this can be a soft cross entropy (Ratner et al., 2016b)). Alfred focuses on the first two stages and aim to efficiently incorporate modern large pretrained models into the development workflow.

Prompting for Pre-Trained Models. With the emergence of large pre-trained langugae

[2] grpc.io

and vision-language models, prompting has become a popular approach to many few-shot and zero-shot tasks (Brown et al., 2020; Schick and Schütze, 2021a; Radford et al., 2021a). Prompting can create training examples, generate data, modify datasets, and improve model reasoning (Schick and Schütze, 2021b; Ye et al., 2022; Chia et al., 2022; Wu et al., 2022; Wang et al., 2022; Wei et al., 2022; Zelikman et al., 2022). This presents a unique opportunity for combining prompting for large pretrained models and weak supervision. Recent studies have investigated strategies to combine large language models into weak supervision frameworks (Smith et al., 2022; Chen et al., 2022; Arora et al., 2022; Zhang et al., 2022b). Alfred aims to provide a platform for the rapid development of weak supervision applications that rely on large pre-trained language and vision-language models, as well as enable experimentation with new ways of using those models.

Systems for Prompt Development. Prompting has led to the development of software toolkits that aid in prompting and research across various tasks. Many tools have been developed for various use cases with large language models. PromptSource (Bach et al., 2022b) is a development environment for creating and archiving sets of prompts. OpenPrompt (Ding et al., 2022) is a library focused on tuning prompts and prompted models with training data. Manifest (Orr, 2022) provides a unified front end for prompting large language models via different APIs. LangChain (Chase, 2022) provides convenient utilities for building applications that chain together multiple prompts and outputs. To complement the existing tools in this growing space, Alfred is designed to be a PWS system based on prompting both large language and vision-language models.

4.3 Prompted LF Development

Alfred is designed to enable the development and application of prompted labeling functions (LFs) Compared to the typical workflow of PWS systems, where developing LFs is not computationally demanding, developing prompted LFs has model inference as a bottleneck. These large models are often hosted remotely on virtual instances or

computing clusters, which can add to the challenge of iterative prompt development. In an iterative development environment, creating prompted labeling functions requires a platform that provides optimal throughput and low latency for a rapid local development experience. To illustrate Alfred's key focuses, we illustrate a typical workflow (Figure 4.2) for using Alfred to create a training dataset:

Step 1: Task Setup For a large model to be used in the development loop, developers can elect to use either self-hosted models or API-based models. For self-hosted models, Alfred provides an `AlfredServer` to host the model on cloud or cluster nodes. As the main development interface, the user can simply start a `Client` by specifying the type of model. Before creating prompted LFs, users need to familiarize themselves with the task by exploring the raw, unlabeled dataset. If there is no development subset available, the developer can annotate a small portion of the data as a held-out evaluation benchmark. Alfred implements a `Dataset` class based on Apache Arrow for fast data access. User may load a `Dataset` from CSV, JSON or Dataframe objects. It also offers direct support for datastes from the Hugging Face 'Dataset' package (Lhoest et al., 2021). During the exploration process, developers may gain insights into the data and the label space, and identify potentially useful heuristics. Moreover, users can freely experiment with prompts with a few unlabeled instances by directly interacting with the `Client`.

Step 2: Iterative Prompt Development: When the user is ready for prompt development, they can use a `Template` to define a prompted LF for either text completion or scoring schemes. A `Template`, given a `Dataset`, will produce the corresponding `Query` objects. `Client` will return the appropriate `Response` object for each `Query`. To map the model responses to votes, users define the corresponding `Voter` and identify the label maps and matching functions to be used for each prompted LFs. Label maps define how potential model responses are associated with the label space. Matching functions specify how the `Voter` determines a match. By default, Alfred employs an exact match mechanism, but this can be substituted with user-defined matching functions for uncased

matching or embedding similarity matching, etc. A `Voter` can be optionally calibrated to reduce model-specific biases (Zhao et al., 2021). With the model responses and the Voter, users can obtain the label votes for each of their prompted LFs and examples in a matrix format. Finally, users can evaluate the quality of their prompted LFs using a set-aside development `Dataset` with desired metrics. Here, users can continue to refine their prompted LFs and iterate as necessary. Once users are satisfied with the performance of their development benchmark, they may proceed.

Step 3: Aggregate Prompt Responses Finally, Alfred can aggregate the votes from each `Voter` with a `LabelModel` to produce probabilistic estimates of the true labels for the examples. Alfred also supports *partial labels*, i.e., labels that narrow down the possible set of classes but are not specific enough to vote on a single class (Yu et al., 2022). The probabilistic labels can then be used to train a wide range of end models.

4.4 System Design

In this section, we describe and highlight the key design decisions for Alfred.

4.4.1 Query and Response Types

We identify two main patterns using prompts for PWS: text completion and scoring. Text completion is when a language model generates responses using a heuristic decoding strategy over the whole model vocabulary, while scoring is when a language ranks candidate completions or a vision-language model ranks candidate prompts, i.e., captions. Alfred implements typed `Query` and `Response` classes for these two patterns (Figure 4.3). Upon applying the `Template` operation on a dataset instance, it produces either a `CompletionQuery` or a `RankedQuery` for each instance based on the `Template` definition. The resulting query can be directly fed into the `Client`. The `Client` then returns a corresponding `CompletionResponse` or `RankedResponse` with the prediction

```
from alfred import Client
from alfred.fm.query import RankedQuery,
                            CompletionQuery
LMClient = Client(...)

headline = "Liverpool wins 7-0!"

LMClient(
    CompletionQuery(headline
    + " What is the topic of this headline?")
)
# Example Response:
# >> CompletionResponse(prediction="Football")

LMClient(
    RankedQuery(headline
    + " What is the topic of this headline?",
      candidate=["Sports", "Politics",
                 "Tech", "Business"])
)
# Example Response:
# >> RankedResponse(prediction="Sports",
#      scores={"Sports":0.76, "Politics":0.10,
#              "Tech":0.07, "Business":0.07 })
```

Figure 4.3: Typed `Query` and `Response` in Alfred

as the main payload, along with any other requested or useful information, such as the logits for each candidate.

4.4.2 Templates for Prompted LFs

Prompt templates are at the core of systems for prompting. In Alfred, prompt templates are expressed as `Template` objects. For natural language tasks, users use the `StringTemplate` class. To produce `Query` objects, users can call 'Template.apply(instance)' or 'Template.apply_to_dataset(dataset).' A `StringTemplate` is defined with a template string with keywords enclosed by double square brackets, e.g. "[[text]] Does the previous context express spouse relation between [[entity_a]] and [[entity_b]]?". An optional field for `Template` is 'answer_choices,' where one may specify the candidate completions. By

```python
from alfred.template import StringTemplate, ImageTemplate

string_template = StringTemplate(
    "Context: [[text]]\n\nIs the above text about weather?",
    answer_choices = ["Yes", "No"]
)
example = {'text': "A pleasant day with a sunny sky."}
prompt = string_template.apply(example)
# >> RankedQuery("""Context: A pleasant day with a sunny sky.
#                   Is the above text about weather?""",
#                 candidates=["Yes", "No"])

image_template = ImageTemplate(
    {"label": ["cat", "dog"]},
    template = "A photo of [[label]]."
)
example = cat_image
prompt = image_template.apply(example)
# >> RankedQuery(example, candidates=["A photo of cat.", "A photo of dog."])
```

Figure 4.4: Example code snippet for creating a `RankedQuery` from a `StringTemplate` or a `ImageTemplate`.

specifying the 'answer_choices,' the `StringTemplate` would yield a `RankedQuery`. An example code snippet showing the creation of a `RankedQuery` is in Figure 4.4. For image annotation tasks, users may define an `ImageTemplate` by specifying the candidate prompts. Upon applying to images, `ImageTemplate` will produce `RankedQuery` objects with the images and candidate prompts.

4.4.3 Throughput Optimization

Alfred is designed to handle large numbers of queries. Self-hosted models from the Transformers package (Wolf et al., 2020) are set up to use model parallelization enabled by Accelerate (Sylvain Gugger, 2022), with user-customizable device maps for parallelizing the model across multiple GPUs. Alfred adopts a dynamic batching strategy that groups instances with similar lengths together and adjusts the input batch size dynamically to maximize model inference throughput. The core idea of the dynamic batching strategy is to group input instances with similar token lengths to minimize padding and maximize

memory utilization.

With the dynamic batching strategy, on a node with 8 NVIDIA Tesla V100s, Alfred achieves a speedup of up to 2.5× and a token throughput increase of 2.9× for approximately 500 queries (∼21,000 tokens) compared to an unoptimized strategy with T0++ (**?**), an 11-billion parameter T5-based (Raffel et al., 2020) language model in FP32. Additionally, Alfred includes a client-side query caching system that automatically stores and updates model responses to facilitate prompt development and avoid redundant queries during development. Alfred also implements a server-side caching system for large multi-modal pretrained models such as CLIP. At inference time, Alfred will cache the input data with its corresponding encoded latent representations from different encoder head for each modalities. This server-side caching system effectively avoids redundant encoding computation on the server end.

4.4.4 Remote Self-Hosting of Models

The computational demands of large pre-trained models can pose a challenge when using them for weak supervision development. To address this challenge, Alfred provides utilities for deploying and interacting with models on remote virtual instances or computing clusters. Additionally, Alfred implements a SSH-based tunneling service that ensures a secure local connection while preserving all Alfred functionality. The tunneling utility also simplifies deployment of the server on shared computing clusters, with the login node serving as a jump host for the computing node. This is particularly useful for using Alfred on centrally-managed shared computing clusters such as those at universities. Alfred's built-in SSH tunneling is also capable of handling 2-factor authentication, which is common for shared clusters. To enable efficient communication between the client and server, Alfred uses gRPC, a high-performance, open-source remote procedure call framework. This enables Alfred to provide a seamless development experience for weak supervision development without the need for expensive local resources.

4.4.5 Mapping Responses to Votes

Another core piece of the Alfred system is the `Voter` class. Each `Voter` defines how to map model responses to votes for the true label of an exmaple. The votes can be class labels or partial labels (e.g., attributes) specified by the users. The voting mechanism also relies on a matching function, which by default only casts a vote for an exact match. Users may provide their intended matching mechanisms such as uncased matching or embedding similarity matching for more flexibility for each `Voter`. Furthermore, `Voter` can be contextually calibrated for the specific `Template` class to reduce model bias towards predicting certain answers. Recent studies show calibration can be helpful for many prompt-based tasks (Zhao et al., 2021; Smith et al., 2022). By calling 'Client.calibrate(Template, Voter),' Alfred will calibrate the voting weights according to the strategy proposed by Zhao et al. (2021) and automatically apply the calibration during voting.

4.4.6 Label Models for Aggregating Votes

Alfred currently includes four label models for combining the votes from prompted labeling functions. The four label models are available to meet different use cases. The `MajorityVote` model is a baseline option suitable for fast development iteration, while the `NaiveBayes` model is recommended as the standard label model. Alfred also includes `NPLM` (Yu et al., 2022) (noisy partial label model) to support weak supervision from partial labels, which are labels that narrow down the possible set of classes but are not specific enough to vote on a single class. `FlyingSquid` (Fu et al., 2020) is the fourth model option and is recommended when `MajorityVote` is not accurate enough but more speed than `NaiveBayes` is needed. These label model classes have a unified interface, providing a consistent experience for users. After processing votes, the label model module generates probabilistic labels, represented as a distribution over the label space, for the given unlabeled dataset. Finally users can use the estimated probabilistic labels to train an end model for the downstream task.

4.5 Example Use Cases

In this section, we present two example use cases for how Alfred can be used to create training data for specific machine learning tasks using natural language prompts in both text and image domains. We measure the labeling quality by taking the top-1 accuracy of the estimated probabilistic labels given by the label model. The notebooks to reproduce these examples are in the Alfred repository.

4.5.1 Youtube Comment Spam Detection

Zero Shot	Prompted LFs	Prompted LFs+C
46.8	57.8	65.3

Table 4.1: Top-1 accuracy on YouTube spam detection. Zero Shot refers to prompting T0++ directly. +C means applying contextual calibration on the `Voter` objects.

In this experiment, we use Alfred to annotate the training split of YouTube spam detection dataset. (Alberto et al., 2015) We replicate the setup used by smith2022language The prompts are translated from the code-based labeling functions provided by the WRENCH benchmark (Zhang et al., 2021a), a comprehensive weak supervision benchmark. Alfred also includes a `WrenchBenchmarkDataset` abstraction for easily running this benchmark. In total, we define 10 prompted labeling functions with `StringTemplate` objects. Responses are mapped to votes using `Voter` objects. For this experiment, we use T0++ (Sanh et al., 2022) as the backbone model for Alfred. Following smith2022language, we also calibrate the responses from T0++ when voting using the contextual calibration strategy proposed by zhao-2022-auto. Finally we aggregate the votes using the `NaiveBayes` label model to produce the probablistic labels. Table 4.1 shows that Alfred makes reproducing the results of smith2022language easy, demonstrating that the combination of weak supervision and calibration yield a dramatic improvement over zero-shot prompting alone.

Zero Shot	Prompted LFs
86.0	92.4

Table 4.2: Top-1 accuracy on Oxford-IIIT Pet breed classification. Zero Shot refers to prompting CLIP directly.

4.5.2 Pet Breed Classification

Traditionally, programmatic weak supervision for vision has been limited by the ability to express supervision in code, relying on models such as object or attribute detectors to extract features and classify. However, these object detectors often depend heavily on supervised training data, becoming a bottleneck for applying programmatic weak supervision in various vision tasks. Fortunately, with large-pretrained vision-language model like CLIP (Radford et al., 2021a), we are now able to express supervision with natural language. For this task, we develop prompts to classify 37 different breeds of pets from the Oxford-IIIT Pet dataset (Parkhi et al., 2012). We use CLIP-ViT/L-14 as the backbone model and developed three simple prompted LFs. The first two prompted LFs use templates "a photo of [[label]]" and "a photo of [[label]] [cat/dog]" where "[cat/dog]" is selected based on the breed. The third prompted LF produces a partial label with the template "a photo of [cat/dog]", encouraging fine-grained labels to match with the coarse-grained type detected by CLIP. We combine the votes using the `NPLM` label (Yu et al., 2022) to support weak supervision at various levels of granularity in the label space. Table 4.2 shows that this multi-granular weak supervision provides a nice boost over zero-shot prompting alone.

4.6 Conclusion

In this chapter, I addressed the limitation of technical barriers to knowledge transfer, where current systems often require domain experts to have programming skills to encode their knowledge into rigid, code-based labeling functions. I introduced *Alfred*, a prototype system designed to enhance programmatic weak supervision by leveraging large pre-

trained models, including large language models and vision-language models. Alfred enables domain experts to express their domain-specific knowledge and heuristics through flexible natural language prompts, eliminating the need for programming expertise and making the labeling process more accessible.

By reducing the manual workload and lowering technical barriers, Alfred empowers domain experts to effectively transfer their knowledge into data labeling tasks within weak supervision frameworks to facilitate efficient knowledge transfer from experts to models. Alfred's optimized inference throughput, smooth local development experience, and compatibility with vision-language models for image annotation tasks further enhance the practical application of weak supervision.

CHAPTER 5

Proposed Work: Advancing Prompted Weak Supervision with Retrieval and Agentic Workflow

5.1 Alfred-Apps: A Platform for Benchmarking Prompted Weak Supervision

Advancing prompted weak supervision systems requires a reliable and extensible platform that supports standardized evaluation and provides accessible application templates for weak supervision practitioners. To meet these needs, I will introduce Alfred-Apps, a benchmarking platform built on top of the prototype prompted weak supervision system, Alfred (Yu and Bach, 2023) 4.

The current prototype of Alfred-Apps supports six text classification tasks with plan to integrate more tasks and provides a streamlined framework for the entire prompted weak supervision workflow, covering all stages from prompt labeler development to end-model training. For each task, I will develop prompted labelers in two categories: (1)

zero-shot labelers, where the language model is prompted to select a label directly from the defined label space without additional training, and (2) heuristic labelers, which apply high-precision rules to assign one or more labels when confident, abstaining otherwise. This platform also allows users to configure new tasks with custom data sources, ensuring that evaluation experimentation remains flexible while following standardized procedures.

Alfred-Apps is built with modularity and extensibility in mind, enabling researchers to customize components without affecting the core functionality of the platform. It includes a comprehensive suite of evaluation metrics—such as precision, recall, and F1-score—along with weak supervision-specific metrics like conflict rates, overlap rates, and coverage for the prompt labelers. These metrics can provide detailed insights into individual labeler performance, helping users understand the reliability and effectiveness of different weak supervision strategies.

I also plan to expand Alfred-Apps to support multimodal tasks, including vision, audio, and video data, allowing users to assess whether techniques developed for text-based tasks generalize effectively to other classification tasks.

Ultimately, Alfred-Apps aims to accelerate research and development in prompted weak supervision by providing a robust framework for evaluating new techniques and facilitating data annotation projects through a prompted weak supervision workflow. Through these capabilities, I believe that Alfred-Apps can make prompted weak supervision both more accessible and more practical for a range of research and real-world applications.

5.2 Alfred Retriever: Integrating Retrieval-Augmented Generation into Prompted Weak Supervision

Retrieval-augmented generation (RAG) has emerged as a powerful paradigm for enhancing large language models (LLMs) with external knowledge (Lewis et al., 2020; Khandelwal et al., 2019). By grounding LLM outputs in retrieved factual information, RAG approaches

have demonstrated significant improvements in accuracy and reliability across various applications (Izacard et al., 2023; Khattab et al., 2022). This trend has particular relevance for prompted weak supervision, where ensuring factual correctness of generated labels is crucial, especially in high-stakes domains such as healthcare, legal, and financial services.

Building on these advances in RAG, I propose to implement Alfred Retriever, a system that enhances the Alfred (Yu and Bach, 2023) 4 framework by incorporating retrieval-augmented generation techniques to ground weak supervision in verifiable external knowledge. This integration addresses a significant challenge in prompted weak supervision: the potential for LLMs to produce hallucinations generating factually incorrect or misleading labels—which can severely impact model reliability in critical applications.

Alfred Retriever extends the prompted weak supervision paradigm in three key ways. (1) it augments LLM-based prompted labeling functions with a retrieval mechanism that accesses domain-specific knowledge bases during the labeling process. (2) it implements a dynamic prompt construction system that incorporates retrieved context to guide LLM outputs. (3) it provides a validation framework that cross-references generated labels against retrieved factual information to ensure consistency and accuracy. The core architecture of Alfred Retriever centers around a modular retrieval pipeline that interfaces with both external knowledge sources and the base Alfred system.

Alfred Retriever will represent a firm step toward more reliable prompted weak supervision by grounding LLM outputs in verifiable external knowledge. This advancement is particularly crucial for safety-critical applications where label accuracy directly impacts system reliability. By combining the flexibility of LLM-based weak supervision with the factual grounding of retrieval-augmented generation, Alfred Retriever is able to address a fundamental challenge in developing reliable machine learning systems.

5.2.1 Evaluation Plan

The evaluation of Alfred Retriever will focus on its impact on label accuracy and factual consistency in safety-critical tasks requiring domain expertise, such as medical and legal classification tasks on Alfred-Apps. Two core areas will be assessed: (1) enhancements in label reliability through retrieval-augmented generation (RAG) and (2) the efficiency and scalability of the retrieval process within the weak supervision pipeline.

For label reliability, I will compare labels generated by Alfred Retriever to those produced by standard LLM-based methods. For retrieval efficiency and scalability, I will evaluate retrieval latency to understand the trade-offs inherent to retrieval-based systems.

All experiments will utilize standardized datasets and thorough documentation to ensure reproducibility. This evaluation is designed to demonstrate that Alfred Retriever effectively grounds LLM-generated labels in external knowledge, enhancing reliability for critical applications.

5.3 AgentAlfred: A LLM-based Agentic Interface for Prompted Weak Supervision

Domain experts often encounter substantial technical barriers when attempting to encode their specialized knowledge into machine learning systems (Ratner et al., 2017). The challenge of translating domain expertise into computational forms remains a key bottleneck in developing effective machine learning solutions.

To reduce these barriers, I propose to implement AgentAlfred, an LLM-based agentic system that proactively assists in developing prompted weak supervision workflows. Drawing inspiration from recent advances in LLM-powered agents (Zhou et al., 2023), AgentAlfred acts as an intelligent collaborator that actively guides domain experts through the process of knowledge encoding.

Built on the Alfred infrastructure (Yu and Bach, 2023) 4, AgentAlfred will be able to manage prompt execution, query handling, and batch processing in the backend. AgentAlfred transforms the traditional programming-based workflow into an interactive, dialogue-driven experience where the system takes initiative in refining and formalizing expert knowledge, similar to recent developments in conversational AI systems.

AgentAlfred leverages large language models (LLMs) not just as translation tools, but as the foundation for an autonomous agent that anticipates expert needs, suggests potential labeling strategies, and identifies edge cases that require attention (**?**). This proactive approach allows domain experts to communicate their insights while the system actively works to structure and validate their knowledge. For example, while developing a sentiment analysis model for movie reviews, rather than passively accepting expert input, AgentAlfred engages in a collaborative dialogue—suggesting potential patterns, questioning assumptions, and proposing refinements based on the data characteristics. Through this dynamic interaction, AgentAlfred can actively help transform intuitive domain expertise into robust prompted weak supervision strategies.

The system architecture includes three key components:

- A natural language-centric interface that enables natural language interactions
- An exploratory data platform that provides real-time feedback
- A backend that translates conversational input into executable weak supervision workflows

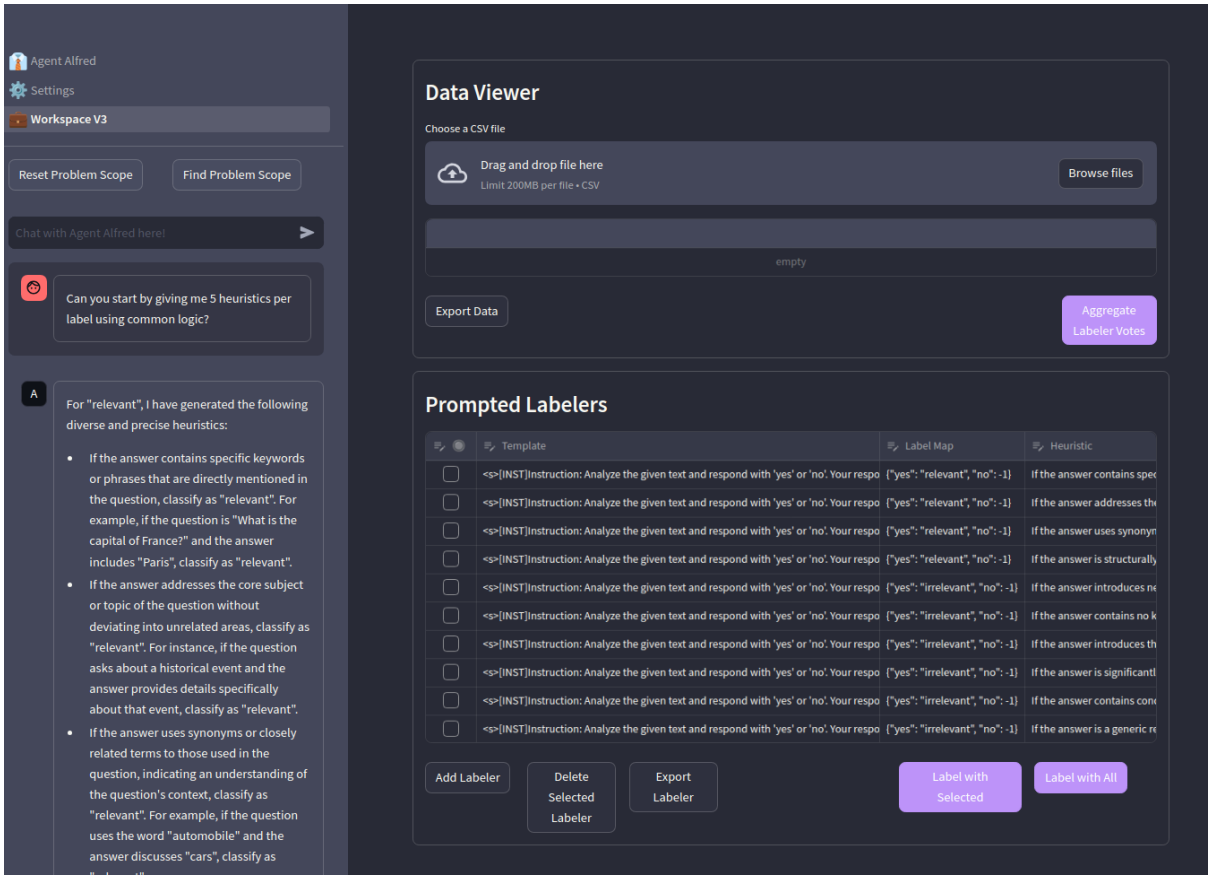

Figure 5.1: Prototype GUI of AgentAlfred: AgentAlfred's interface combines natural language interaction with real-time data exploration.

The current prototype GUI demonstrates the feasibility of this approach. The main interface includes a persistent chat window alongside data and prompted labeler view panels, allowing experts to seamlessly switch between expressing their knowledge and examining dataset patterns. This design promotes rapid iteration between knowledge expression and result verification, streamlining the creation of effective prompted weak supervision pipelines.

AgentAlfred will represent an important advancement in making weak supervision more accessible to domain experts. By removing technical barriers and enabling natural knowledge transfer, AgentAlfred can bring the full potential of expert insights closer to realization in machine learning systems development.

5.3.1 Evaluation Plan

The evaluation of AgentAlfred will focus on two core aspects: its effectiveness as an interactive, agentic interface and its capability to facilitate complete prompted weak supervision workflows. I will conduct both quantitative and qualitative evaluations to assess (1) the quality of agent interaction and (2) the end-to-end performance of the system with various large language model (LLM) backbones.

To evaluate interaction quality, experiments will be conducted across five representative tasks from Alfred-Apps. Key metrics will include the (1) instruction success rate, which records how accurately AgentAlfred interprets and executes user commands. (2) the perceived usefulness of system-generated suggestions, which tracked via a binary feedback system embedded within the AgentAlfred interface.

For end-to-end performance evaluation, I will assess the overall efficiency of the workflow from the initial expert interaction to final model deployment within Alfred-Apps. This will include metrics such as time to deployment, model accuracy, and resource efficiency. To ensure reproducibility and comparability, all evaluations will use standardized datasets and detailed documentation of experimental procedures.

Through these evaluations, I aim to demonstrate AgentAlfred's potential to improve both user interaction quality and workflow efficiency, ultimately making weak supervision more accessible and effective for users across diverse applications.

References

Alberto, T. C., Lochter, J. V., and Almeida, T. A. (2015). Tubespam: Comment spam filtering on youtube. In *2015 IEEE 14th international conference on machine learning and applications (ICMLA)*, pages 138–143. IEEE.

Allman, E. S., Matias, C., Rhodes, J. A., et al. (2009). Identifiability of parameters in latent structure models with many observed variables. *The Ann. of Stat.*, 37(6A):3099–3132.

Allman, E. S., Rhodes, J. A., Stanghellini, E., and Valtorta, M. (2015). Parameter identifiability of discrete bayesian networks with hidden variables. *J. of Causal Inference*, 3(2):189–205.

Arachie, C. and Huang, B. (2021). A general framework for adversarial label learning. *J. of Mach. Learn. Research*, 22:1–33.

Arora, S., Narayan, A., Chen, M. F., Orr, L. J., Guha, N., Bhatia, K., Chami, I., Sala, F., and Ré, C. (2022). Ask me anything: A simple strategy for prompting language models. *arXiv preprint arXiv:2210.02441*.

Awasthi, A., Ghosh, S., Goyal, R., and Sarawagi, S. (2020). Learning from rules generalizing labeled exemplars. In *ICLR*.

Bach, S. H., He, B., Ratner, A., and Ré, C. (2017). Learning the structure of generative models without labeled data. In *ICML*.

Bach, S. H., Rodriguez, D., Liu, Y., Luo, C., Shao, H., Xia, C., Sen, S., Ratner, A., Hancock, B., Alborzi, H., Kuchhal, R., Ré, C., and Malkin, R. (2019). Snorkel DryBell: A case study in deploying weak supervision at industrial scale. In *SIGMOD*.

Bach, S. H., Sanh, V., Yong, Z.-X., Webson, A., Raffel, C., Nayak, N. V., Sharma, A., Kim, T., Bari, M. S., Fevry, T., Alyafeai, Z., Dey, M., Santilli, A., Sun, Z., Ben-David, S., Xu, C., Chhablani, G., Wang, H., Fries, J. A., Al-shaibani, M. S., Sharma, S., Thakker, U., Almubarak, K., Tang, X., Radev, D., Jiang, M. T.-J., and Rush, A. M. (2022a). PromptSource: An integrated development environment and repository for natural language prompts. In *Meeting of the Association for Computational Linguistics (ACL) Demonstration*.

Bach, S. H., Sanh, V., Yong, Z.-X., Webson, A., Raffel, C., Nayak, N. V., Sharma, A., Kim, T., Bari, M. S., Fevry, T., Alyafeai, Z., Dey, M., Santilli, A., Sun, Z., Ben-David, S., Xu, C., Chhablani, G., Wang, H., Fries, J. A., Al-shaibani, M. S., Sharma, S., Thakker, U., Almubarak, K., Tang, X., Tang, X., Jiang, M. T.-J., and Rush, A. M. (2022b). Promptsource: An integrated development environment and repository for natural language prompts.

Balsubramani, A. and Freund, Y. (2015). Optimally combining classifiers using unlabeled data. In *COLT*.

Balsubramani, A. and Freund, Y. (2016a). Optimal binary classifier aggregation for general losses. In *NeurIPS*.

Balsubramani, A. and Freund, Y. (2016b). Scalable semi-supervised aggregation of classifiers. In *NeurIPS*.

Bar, A., Herzig, R., Wang, X., Rohrbach, A., Chechik, G., Darrell, T., and Globerson, A. (2021). Compositional video synthesis with action graphs. In *ICML*.

Ben-Zaken, E., Ravfogel, S., and Goldberg, Y. (2022). Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In *ACL*.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., Bernstein, M. S., Bohg, J., Bosselut, A., Brunskill, E., Brynjolfsson, E., Buch, S., Card, D., Castellon, R., Chatterji, N. S., Chen, A. S., Creel, K., Davis, J., Demszky, D., Donahue, C., Doumbouya, M., Durmus, E., Ermon, S., Etchemendy, J., Ethayarajh, K., Fei-Fei, L., Finn, C., Gale, T., Gillespie, L. E., Goel, K., Goodman, N. D., Grossman, S., Guha, N., Hashimoto, T., Henderson, P., Hewitt, J., Ho, D. E., Hong, J., Hsu, K., Huang, J., Icard, T. F., Jain, S., Jurafsky, D., Kalluri, P., Karamcheti, S., Keeling, G., Khani, F., Khattab, O., Koh, P. W., Krass, M. S., Krishna, R., Kuditipudi, R., Kumar, A., Ladhak, F., Lee, M., Lee, T., Leskovec, J., Levent, I., Li, X. L., Li, X., Ma, T., Malik, A., Manning, C. D., Mirchandani, S. P., Mitchell, E., Munyikwa, Z., Nair, S., Narayan, A., Narayanan, D., Newman, B., Nie, A., Niebles, J. C., Nilforoshan, H., Nyarko, J. F., Ogut, G., Orr, L., Papadimitriou, I., Park, J. S., Piech, C., Portelance, E., Potts, C., Raghunathan, A., Reich, R., Ren, H., Rong, F., Roohani, Y. H., Ruiz, C., Ryan, J., R'e, C., Sadigh, D., Sagawa, S., Santhanam, K., Shih, A., Srinivasan, K. P., Tamkin, A., Taori, R., Thomas, A. W., Tramèr, F., Wang, R. E., Wang, W., Wu, B., Wu, J., Wu, Y., Xie, S. M., Yasunaga, M., You, J., Zaharia, M. A., Zhang, M., Zhang, T., Zhang, X., Zhang, Y., Zheng, L., Zhou, K., and Liang, P. (2021). On the opportunities and risks of foundation models. *ArXiv*, abs/2108.07258.

Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. (2020). Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H., editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.

Bucher, M., Herbin, S., and Jurie, F. (2017). Generating visual representations for zero-shot classification. In *ICCV Workshops*.

Cabannes, V., Bach, F., and Rudi, A. (2021). Disambiguation of weak supervision with exponential convergence rates. *arXiv preprint arXiv:2102.02789*.

Cabannnes, V., Rudi, A., and Bach, F. (2020). Structured prediction with partial labelling through the infimum loss. In *ICML*.

Changpinyo, S., Chao, W.-L., Gong, B., and Sha, F. (2016). Synthesized classifiers for zero-shot learning. In *CVPR*.

Chao, W.-L., Changpinyo, S., Gong, B., and Sha, F. (2016). An empirical study and

analysis of generalized zero-shot learning for object recognition in the wild. In *European conference on computer vision (ECCV)*.

Chase, H. (2022). LangChain.

Chen, M. F., Fu, D. Y., Adila, D., Zhang, M., Sala, F., Fatahalian, K., and Ré, C. (2022). Shoring up the foundations: Fusing model embeddings and weak supervision. In *Uncertainty in Artificial Intelligence*, pages 357–367. PMLR.

Chen, V. S., Varma, P., Krishna, R., Bernstein, M., Re, C., and Fei-Fei, L. (2019). Scene graph prediction with limited labels. In *ICCV*.

Chia, Y. K., Bing, L., Poria, S., and Si, L. (2022). Relationprompt: Leveraging prompts to generate synthetic data for zero-shot relation triplet extraction. *arXiv preprint arXiv:2203.09101*.

Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on information theory*, 2(3):113–124.

Cohan, A., Ammar, W., Van Zuylen, M., and Cady, F. (2019). Structural scaffolds for citation intent classification in scientific publications. In *NAACL*.

Cour, T., Sapp, B., and Taskar, B. (2011). Learning from partial labels. *The J. of Mach. Learn. Research*, 12:1501–1536.

Dawid, A. P. and Skene, A. M. (1979). Maximum likelihood estimation of observer error-rates using the EM algorithm. *Journal of the Royal Statistical Society C*, 28(1):20–28.

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). Bert: Pre-training of deep bidirectional transformers for language understanding. In *NAACL*.

Ding, N., Hu, S., Zhao, W., Chen, Y., Liu, Z., Zheng, H., and Sun, M. (2022). OpenPrompt: An open-source framework for prompt-learning. In Basile, V., Kozareva, Z., and Stajner, S., editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 105–113, Dublin, Ireland. Association for Computational Linguistics.

Dong, X., Bao, J., Zhang, T., Chen, D., Shuyang, G., Zhang, W., Yuan, L., Chen, D., Wen, F., and Yu, N. (2022). Clip itself is a strong fine-tuner: Achieving 85.7 *arXiv preprint arXiv:2212.06138*.

Drozdov, A., Schärli, N., Akyürek, E., Scales, N., Song, X., Chen, X., Bousquet, O., and Zhou, D. (2022). Compositional semantic parsing with large language models. *arXiv preprint arXiv:2209.15003*.

Durand, T., Mehrasa, N., and Mori, G. (2019). Learning a deep convnet for multi-label classification with partial labels. In *CVPR*.

Farhadi, A., Endres, I., Hoiem, D., and Forsyth, D. A. (2009). Describing objects by their attributes. In *CVPR*, pages 1778–1785. IEEE Computer Society.

Felix, R., Reid, I., Carneiro, G., et al. (2018). Multi-modal cycle-consistent generalized zero-shot learning. In *ECCV*.

Feng, L., Kaneko, T., Han, B., Niu, G., An, B., and Sugiyama, M. (2020a). Learning with multiple complementary labels. In *ICML*.

Feng, L., Lv, J., Han, B., Xu, M., Niu, G., Geng, X., An, B., and Sugiyama, M. (2020b). Provably consistent partial-label learning. In *NeurIPS*.

Feng, L., Lv, J., Han, B., Xu, M., Niu, G., Geng, X., An, B., and Sugiyama, M. (2020c). Provably consistent partial-label learning.

Fodor, J. A. and Pylyshyn, Z. W. (1988). Connectionism and cognitive architecture: A critical analysis. *Cognition*, 28:3–71.

Fries, J., Varma, P., Chen, V., Xiao, K., Tejeda, H., Priyanka, S., Dunnmon, J., Chubb, H., Maskatia, S., Fiterau, M., Delp, S., Ashley, E., Ré, C., and Priest, J. (2019). Weakly supervised classification of rare aortic valve malformations using unlabeled cardiac MRI sequences. *Nature Communications*, 10(1).

Fries, J. A., Steinberg, E., Khattar, S., Fleming, S. L., Posada, J., Callahan, A., and Shah, N. H. (2021). Ontology-driven weak supervision for clinical entity classification in electronic health records. *Nature Communications*, 12(1):1–11.

Frome, A., Corrado, G. S., Shlens, J., Bengio, S., Dean, J., Ranzato, M., and Mikolov, T. (2013). Devise: A deep visual-semantic embedding model. In *NeurIPS*.

Fu, D. Y., Chen, M. F., Sala, F., Hooper, S. M., Fatahalian, K., and Ré, C. (2020). Fast and three-rious: Speeding up weak supervision with triplet methods. In *Proceedings of the 37th International Conference on Machine Learning (ICML 2020)*, pages 3280–3291. PMLR.

Gao, C. and Zhou, D. (2013). Minimax optimal convergence rates for estimating ground truth from crowdsourced labels. *CoRR*, abs/1207.0016.

Ge, C., Huang, R., Xie, M., Lai, Z., Song, S., Li, S., and Huang, G. (2022). Domain adaptation via prompt learning. *ArXiv preprint*, abs/2202.06687.

Gulli, A. (2005). *AG's corpus of news articles*.

Guo, D., Rush, A., and Kim, Y. (2021). Parameter-efficient transfer learning with diff pruning. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 4884–4896, Online. Association for Computational Linguistics.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *CVPR*.

Herzig, R., Bar, A., Xu, H., Chechik, G., Darrell, T., and Globerson, A. (2020). Learning canonical representations for scene graph to image generation. In *European Conference on Computer Vision*, pages 210–227. Springer.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., de Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. (2019). Parameter-efficient transfer learning for NLP. In

Chaudhuri, K. and Salakhutdinov, R., editors, *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pages 2790–2799. PMLR.

Hudson, D. A. and Manning, C. D. (2019). GQA: A new dataset for real-world visual reasoning and compositional question answering. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 6700–6709. Computer Vision Foundation / IEEE.

Hüllermeier, E. and Cheng, W. (2015). Superset learning based on generalized loss minimization. In *ECML PKDD*.

Hupkes, D., Dankers, V., Mul, M., and Bruni, E. (2020). Compositionality decomposed: How do neural networks generalise? *J. Artif. Intell. Res.*, 67:757–795.

Ishida, T., Niu, G., Hu, W., and Sugiyama, M. (2017). Learning from complementary labels. In *NeurIPS*.

Isola, P., Lim, J. J., and Adelson, E. H. (2015). Discovering states and transformations in image collections. In *IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015*, pages 1383–1391. IEEE Computer Society.

Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., and Grave, E. (2023). Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43.

Jain, A., Guo, M., Srinivasan, K., Chen, T., Kudugunta, S., Jia, C., Yang, Y., and Baldridge, J. (2021). Mural: multimodal, multitask retrieval across languages. *ArXiv preprint*, abs/2109.05125.

Jayaraman, D. and Grauman, K. (2014). Zero shot recognition with unreliable attributes. In *NeurIPS*.

Jia, C., Yang, Y., Xia, Y., Chen, Y., Parekh, Z., Pham, H., Le, Q. V., Sung, Y., Li, Z., and Duerig, T. (2021). Scaling up visual and vision-language representation learning with noisy text supervision. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 4904–4916. PMLR.

Jin, R. and Ghahramani, Z. (2002). Learning with multiple labels. In *NeurIPS*.

Jin, W., Cheng, Y., Shen, Y., Chen, W., and Ren, X. (2022). A good prompt is worth millions of parameters: Low-resource prompt-based learning for vision-language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Johnson, J., Hariharan, B., Van Der Maaten, L., Fei-Fei, L., Lawrence Zitnick, C., and Girshick, R. (2017). Clevr: A diagnostic dataset for compositional language and

elementary visual reasoning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2901–2910.

Ju, C., Han, T., Zheng, K., Zhang, Y., and Xie, W. (2021). Prompting visual-language models for efficient video understanding. *ArXiv preprint*, abs/2112.04478.

Karamanolakis, G., Mukherjee, S., Zheng, G., and Awadallah, A. H. (2021). Self-training with weak supervision. In *NAACL*.

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., and Lewis, M. (2019). Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172*.

Khattab, O., Santhanam, K., Li, X. L., Hall, D., Liang, P., Potts, C., and Zaharia, M. (2022). Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive nlp. *arXiv preprint arXiv:2212.14024*.

Kruskal, J. B. (1977). Three-way arrays: rank and uniqueness of trilinear decompositions, with application to arithmetic complexity and statistics. *Linear algebra and its applications*, 18(2):95–138.

Lake, B. M. and Baroni, M. (2018). Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In Dy, J. G. and Krause, A., editors, *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pages 2879–2888. PMLR.

Lampert, C. H., Nickisch, H., and Harmeling, S. (2009). Learning to detect unseen object classes by between-class attribute transfer. In *CVPR*.

Lester, B., Al-Rfou, R., and Constant, N. (2021). The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3045–3059, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., et al. (2020). Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

Lhoest, Q., del Moral, A. V., Jernite, Y., Thakur, A., von Platen, P., Patil, S., Chaumond, J., Drame, M., Plu, J., Tunstall, L., et al. (2021). Datasets: A community library for natural language processing. *arXiv preprint arXiv:2109.02846*.

Li, C., Li, X., and Ouyang, J. (2020a). Learning with noisy partial labels by simultaneously leveraging global and local consistencies. In *CIKM*.

Li, X. and Roth, D. (2002). Learning question classifiers. In *COLING*.

Li, X. L. and Liang, P. (2021). Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*

(Volume 1: Long Papers), pages 4582–4597, Online. Association for Computational Linguistics.

Li, Y., Liang, F., Zhao, L., Cui, Y., Ouyang, W., Shao, J., Yu, F., and Yan, J. (2021). Supervision exists everywhere: A data efficient contrastive language-image pre-training paradigm. *ArXiv*, abs/2110.05208.

Li, Y., Xu, Y., Mao, X., and Lu, C. (2020b). Symmetry and group in attribute-object compositions. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pages 11313–11322. IEEE.

Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., and Raffel, C. (2022). Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *ArXiv preprint*, abs/2205.05638.

Liu, L. and Dietterich, T. (2014). Learnability of the superset label learning problem. In *ICML*.

Liu, L. and Dietterich, T. G. (2012). A conditional multinomial mixture model for superset label learning. In *NeurIPS*.

Liu, L., Zhou, T., Long, G., Jiang, J., and Zhang, C. (2020). Attribute propagation network for graph zero-shot learning. In *AAAI*.

Liu, Y., Guo, J., Cai, D., and He, X. (2019). Attribute attention for semantic disambiguation in zero-shot learning. In *ICCV*.

Luo, J. and Orabona, F. (2010). Learning from candidate labeling sets. Technical report.

Mahabadi, R. K., Henderson, J., and Ruder, S. (2021). Compacter: Efficient low-rank hypercomplex adapter layers. In *NeurIPS*.

Mallinar, N., Shah, A., Ugrani, R., Gupta, A., Gurusankar, M., Ho, T. K., Liao, Q. V., Zhang, Y., Bellamy, R. K., Yates, R., et al. (2019). Bootstrapping conversational agents with weak supervision. In *AAAI*.

Mancini, M., Naeem, M., Xian, Y., and Akata, Z. (2021a). Open world compositional zero-shot learning. In *34th IEEE Conference on Computer Vision and Pattern Recognition*. IEEE.

Mancini, M., Naeem, M. F., Xian, Y., and Akata, Z. (2021b). Learning graph embeddings for open world compositional zero-shot learning. *ArXiv*, abs/2105.01017.

Marcus, G. F. (2003). *The algebraic mind: Integrating connectionism and cognitive science*. MIT press.

Mazzetto, A., Cousins, C., Sam, D., Bach, S. H., and Upfal, E. (2021a). Adversarial multiclass learning under weak supervision with performance guarantees. In *ICML*.

Mazzetto, A., Sam, D., Park, A., Upfal, E., and Bach, S. H. (2021b). Semi-supervised aggregation of dependent weak supervision sources with performance guarantees. In *AISTATS*.

Misra, I., Gupta, A., and Hebert, M. (2017). From red wine to red tomato: Composition with context. In *2017 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017, Honolulu, HI, USA, July 21-26, 2017*, pages 1160–1169. IEEE Computer Society.

Mitchell, E., Lin, C., Bosselut, A., Finn, C., and Manning, C. D. (2022). Fast model editing at scale. In *International Conference on Learning Representations*.

Naeem, M. F., Xian, Y., Tombari, F., and Akata, Z. (2021). Learning graph embeddings for compositional zero-shot learning. *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 953–962.

Nagarajan, T. and Grauman, K. (2018). Attributes as operators. *ECCV*.

Narayan, S., Gupta, A., Khan, F. S., Snoek, C. G., and Shao, L. (2020). Latent embedding feedback and discriminative features for zero-shot classification. In *ECCV*.

Nawhal, M., Zhai, M., Lehrmann, A., Sigal, L., and Mori, G. (2020). Generating videos of zero-shot compositions of actions and objects. In *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XII*, page 382–401, Berlin, Heidelberg. Springer-Verlag.

Nguyen, N. and Caruana, R. (2008). Classification with partial labels. In *KDD*.

Nitzan, S. and Paroush, J. (1982). Optimal decision rules in uncertain dichotomous choice situations. *International Economic Review*, 23(2):289–97.

Norouzi, M., Mikolov, T., Bengio, S., Singer, Y., Shlens, J., Frome, A., Corrado, G. S., and Dean, J. (2013). Zero-shot learning by convex combination of semantic embeddings. *arXiv preprint arXiv:1312.5650*.

Orr, L. (2022). Manifest. `https://github.com/HazyResearch/manifest`.

Palatucci, M., Pomerleau, D., Hinton, G. E., and Mitchell, T. M. (2009). Zero-shot learning with semantic output codes. In *NeurIPS*.

Parkhi, O. M., Vedaldi, A., Zisserman, A., and Jawahar, C. (2012). Cats and dogs. In *2012 IEEE conference on computer vision and pattern recognition*, pages 3498–3505. IEEE.

Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., and Lerer, A. (2017). Automatic differentiation in PyTorch. In *NeurIPS Autodiff Workshop*.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. (2019). Pytorch: An imperative style, high-performance deep learning library. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E. B., and Garnett, R., editors, *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pages 8024–8035.

Pennington, J., Socher, R., and Manning, C. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, Doha, Qatar. Association for Computational Linguistics.

Pourpanah, F., Abdar, M., Luo, Y., Zhou, X., Wang, R., Lim, C., and Wang, X. (2020). A review of generalized zero-shot learning methods. *ArXiv*, abs/2011.08641.

Purushwalkam, S., Nickel, M., Gupta, A., and Ranzato, M. (2019). Task-driven modular networks for zero-shot compositional learning. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*, pages 3592–3601. IEEE.

Qin, G. and Eisner, J. (2021). Learning how to ask: Querying LMs with mixtures of soft prompts. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5203–5212, Online. Association for Computational Linguistics.

Radenović, F., Sinha, A., Gordo, A., Berg, T. L., and Mahajan, D. K. (2021). Large-scale attribute-object compositions. *ArXiv*.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. (2021a). Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021b). Learning transferable visual models from natural language supervision. In Meila, M. and Zhang, T., editors, *Proceedings of the 38th International Conference on Machine Learning, ICML 2021, 18-24 July 2021, Virtual Event*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763. PMLR.

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., Sutskever, I., et al. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 21(1):5485–5551.

Ratner, A., Bach, S. H., Ehrenberg, H., Fries, J., Wu, S., and Ré, C. (2017). Snorkel: Rapid training data creation with weak supervision. In *Proceedings of the VLDB Endowment. International Conference on Very Large Data Bases*, volume 11, page 269. NIH Public Access.

Ratner, A. J., De Sa, C. M., Wu, S., Selsam, D., and Ré, C. (2016a). Data programming: Creating large training sets, quickly. In *NeurIPS*.

Ratner, A. J., De Sa, C. M., Wu, S., Selsam, D., and Ré, C. (2016b). Data programming: Creating large training sets, quickly. *Advances in neural information processing systems*, 29.

Ratner, A. J., Hancock, B., Dunnmon, J., Sala, F., Pandey, S., and Ré, C. (2019). Training complex models with multi-task weak supervision. In *AAAI*.

Romera-Paredes, B. and Torr, P. (2015). An embarrassingly simple approach to zero-shot learning. In *ICML*.

Ruis, F., Burghouts, G. J., and Bucur, D. (2021). Independent prototype propagation for zero-shot compositionality. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34.

Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A. C., and Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International J. of Computer Vision*.

Saab, K., Dunnmon, J., Ré, C., Rubin, D., and Lee-Messer, C. (2020). Weak supervision as an efficient approach for automated seizure detection in electroencephalography. *NPJ Digital Medicine*, 3(1):1–12.

Safranchik, E., Luo, S., and Bach, S. H. (2020). Weakly supervised sequence tagging from noisy rules. In *AAAI*.

Sanh, V., Webson, A., Raffel, C., Bach, S., Sutawika, L., Alyafeai, Z., Chaffin, A., Stiegler, A., Raja, A., Dey, M., Bari, M. S., Xu, C., Thakker, U., Sharma, S. S., Szczechla, E., Kim, T., Chhablani, G., Nayak, N., Datta, D., Chang, J., Jiang, M. T.-J., Wang, H., Manica, M., Shen, S., Yong, Z. X., Pandey, H., Bawden, R., Wang, T., Neeraj, T., Rozen, J., Sharma, A., Santilli, A., Fevry, T., Fries, J. A., Teehan, R., Scao, T. L., Biderman, S., Gao, L., Wolf, T., and Rush, A. M. (2022). Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations*.

Sariyildiz, M. B. and Cinbis, R. G. (2019). Gradient matching generative networks for zero-shot learning. In *CVPR*.

Schick, T. and Schütze, H. (2021a). Exploiting cloze-questions for few-shot text classification and natural language inference. In Merlo, P., Tiedemann, J., and Tsarfaty, R., editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 255–269, Online. Association for Computational Linguistics.

Schick, T. and Schütze, H. (2021b). Generating datasets with pretrained language models. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t., editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6943–6951, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Scialom, T., Chakrabarty, T., and Muresan, S. (2022). Fine-tuned language models are continual learners. In *Conference on Empirical Methods in Natural Language Processing*.

Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E., and Singh, S. (2020). AutoPrompt: Eliciting Knowledge from Language Models with Automatically Generated Prompts. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4222–4235, Online. Association for Computational Linguistics.

Smith, R., Fries, J. A., Hancock, B., and Bach, S. H. (2022). Language models in the loop: Incorporating prompting into weak supervision. *arXiv preprint arXiv:2205.02318*.

Socher, R., Ganjoo, M., Manning, C. D., and Ng, A. (2013). Zero-shot learning through cross-modal transfer. In *NeurIPS*.

Song, J., Shen, C., Yang, Y., Liu, Y., and Song, M. (2018). Transductive unbiased embedding for zero-shot learning. In *CVPR*.

Sung, Y.-L., Nair, V., and Raffel, C. (2021). Training neural networks with fixed sparse masks. In *NeurIPS*.

Sylvain Gugger, Lysandre Debut, T. W. P. S. Z. M. S. M. (2022). Accelerate: Training and inference at scale made simple, efficient and adaptable. `https://github.com/huggingface/accelerate`.

Varma, P., Sala, F., He, A., Ratner, A., and Ré, C. (2019). Learning dependency structures for weak supervision models. In *ICML*.

Verma, V. K., Arora, G., Mishra, A., and Rai, P. (2018). Generalized zero-shot learning via synthesized examples. In *CVPR*.

Vu, T., Lester, B., Constant, N., Al-Rfou, R., and Cer, D. M. (2021). Spot: Better frozen model adaptation through soft prompt transfer. *ArXiv*.

Wan, Z., Chen, D., Li, Y., Yan, X., Zhang, J., Yu, Y., and Liao, J. (2019). Transductive zero-shot learning with visual structure constraint. In *NeurIPS*.

Wang, H., Liu, W., Zhao, Y., Hu, T., Chen, K., and Chen, G. (2020). Learning from multi-dimensional partial labels. In *IJCAI*.

Wang, W., Zheng, V. W., Yu, H., and Miao, C. (2019). A survey of zero-shot learning: Settings, methods, and applications. *ACM Transactions on Intelligent Systems and Technology*, 10(2):1–37.

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., and Zhou, D. (2022). Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Chi, E., Le, Q., and Zhou, D. (2022). Chain of thought prompting elicits reasoning in large language models. *arXiv preprint arXiv:2201.11903*.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. (2020). Transformers: State-of-the-art natural language processing. In Liu, Q. and Schlangen, D., editors, *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online. Association for Computational Linguistics.

Wu, Y., Gardner, M., Stenetorp, P., and Dasigi, P. (2022). Generating data to mitigate

spurious correlations in natural language inference datasets. In Muresan, S., Nakov, P., and Villavicencio, A., editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2660–2676, Dublin, Ireland. Association for Computational Linguistics.

Xian, Y., Akata, Z., Sharma, G., Nguyen, Q., Hein, M., and Schiele, B. (2016). Latent embeddings for zero-shot classification. In *CVPR*.

Xian, Y., Lampert, C. H., Schiele, B., and Akata, Z. (2018). Zero-shot learning: A comprehensive evaluation of the good, the bad and the ugly. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(9):2251–2265.

Xian, Y., Sharma, S., Schiele, B., and Akata, Z. (2019). f-vaegan-d2: A feature generating framework for any-shot learning. In *CVPR*.

Xie, M.-K. and Huang, S.-J. (2021). Partial multi-label learning with noisy label identification. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.

Xu, G., Kordjamshidi, P., and Chai, J. (2021). Zero-shot compositional concept learning. In *Proceedings of the 1st Workshop on Meta Learning and Its Applications to Natural Language Processing*, pages 19–27, Online. Association for Computational Linguistics.

Yan, Y. and Guo, Y. (2020). Partial label learning with batch label correction. In *AAAI*.

Ye, J., Gao, J., Li, Q., Xu, H., Feng, J., Wu, Z., Yu, T., and Kong, L. (2022). Zerogen: Efficient zero-shot learning via dataset generation. *arXiv preprint arXiv:2202.07922*.

Ye, Y., Singh, M., Gupta, A., and Tulsiani, S. (2019). Compositional video prediction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10353–10362.

Yu, A. and Grauman, K. (2014). Fine-grained visual comparisons with local learning. In *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*, pages 192–199. IEEE Computer Society.

Yu, P. and Bach, S. (2023). Alfred: A system for prompted weak supervision. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 479–488, Toronto, Canada. Association for Computational Linguistics.

Yu, P., Ding, T., and Bach, S. H. (2022). Learning from multiple noisy partial labelers. In *Artificial Intelligence and Statistics (AISTATS)*.

Yuan, X., Côté, M.-A., Fu, J., Lin, Z., Pal, C., Bengio, Y., and Trischler, A. (2019). Interactive language learning by question answering. *arXiv preprint arXiv:1908.10909*.

Yun, T., Bhalla, U., Pavlick, E., and Sun, C. (2022). Do vision-language pretrained models learn primitive concepts? *arXiv preprint arXiv:2203.17271*.

Zelikman, E., Wu, Y., and Goodman, N. D. (2022). Star: Bootstrapping reasoning with reasoning. *arXiv preprint arXiv:2203.14465*.

Zhang, J., Hsieh, C.-Y., Yu, Y., Zhang, C., and Ratner, A. (2022a). A survey on programmatic weak supervision. *arXiv preprint arXiv:2202.05433*.

Zhang, J., Yu, Y., Li, Y., Wang, Y., Yang, Y., Yang, M., and Ratner, A. (2021a). WRENCH: A comprehensive benchmark for weak supervision. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*.

Zhang, R., Yu, Y., Shetty, P., Song, L., and Zhang, C. (2022b). Prboost: Prompt-based rule discovery and boosting for interactive weakly-supervised learning. *arXiv preprint arXiv:2203.09735*.

Zhang, Z.-R., Zhang, Q.-W., Cao, Y., and Zhang, M.-L. (2021b). Exploiting unlabeled data via partial label assignment for multi-class semi-supervised learning. In *AAAI*.

Zhao, B., Fu, Y., Liang, R., Wu, J., Wang, Y., and Wang, Y. (2019). A large-scale attribute dataset for zero-shot learning. In *CVPR Workshops*.

Zhao, Z., Wallace, E., Feng, S., Klein, D., and Singh, S. (2021). Calibrate before use: Improving few-shot performance of language models. In *International Conference on Machine Learning*, pages 12697–12706. PMLR.

Zhou, K., Yang, J., Loy, C. C., and Liu, Z. (2021). Learning to prompt for vision-language models. *ArXiv preprint*, abs/2109.01134.

Zhou, K., Yang, J., Loy, C. C., and Liu, Z. (2022). Conditional prompt learning for vision-language models. In *CVPR*.

Zhou, W., Jiang, Y. E., Li, L., Wu, J., Wang, T., Qiu, S., Zhang, J., Chen, J., Wu, R., Wang, S., et al. (2023). Agents: An open-source framework for autonomous language agents. *arXiv preprint arXiv:2309.07870*.